

```
In [1]: import seaborn as sns
```

Seaborn is a python data visualisation library it is mostly used for statistical visualisation it build on top of matplotlib and that provides high level API's for visualising the data

Seaborn vs matplotlib

Matplotlib:

Low level plotting

plot from scratch

code length is more because you need to style colors, legends etc

Seaborn:

seaborn is built on top of the matplotlib that means it is the extension of matplotlib

high level plotting

code is less because it is automatically handles the styling, legends, colors

quick data visualisation and working with pandas and dataframes

seaborn installation

pip install seaborn

```
In [2]: sns.__version__
```

```
Out[2]: '0.13.2'
```

```
In [3]: print(sns.get_dataset_names())
```

```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes', 'diamonds',  
'dots', 'dowjones', 'exercise', 'flights', 'fmri', 'geyser', 'glue', 'healthexp', 'i  
ris', 'mpg', 'penguins', 'planets', 'seaice', 'taxis', 'tips', 'titanic']
```

```
In [4]: import numpy as np  
import matplotlib.pyplot as plt  
import pandas as pd
```

seaborn styles

need to use set_style()-->below themes are available

Darkgrid(Default)

Whitegrid

Dark

white

Ticks

```
In [5]: # Load data
```

```
iris=sns.load_dataset('iris')
```

```
In [6]: iris.head()
```

```
Out[6]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [7]: iris.tail()
```

```
Out[7]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

```
In [8]: iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null    float64
1   sepal_width     150 non-null    float64
2   petal_length    150 non-null    float64
3   petal_width     150 non-null    float64
4   species         150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
In [9]: iris.shape
```

```
Out[9]: (150, 5)
```

```
In [10]: print(iris.groupby('species').size())
```

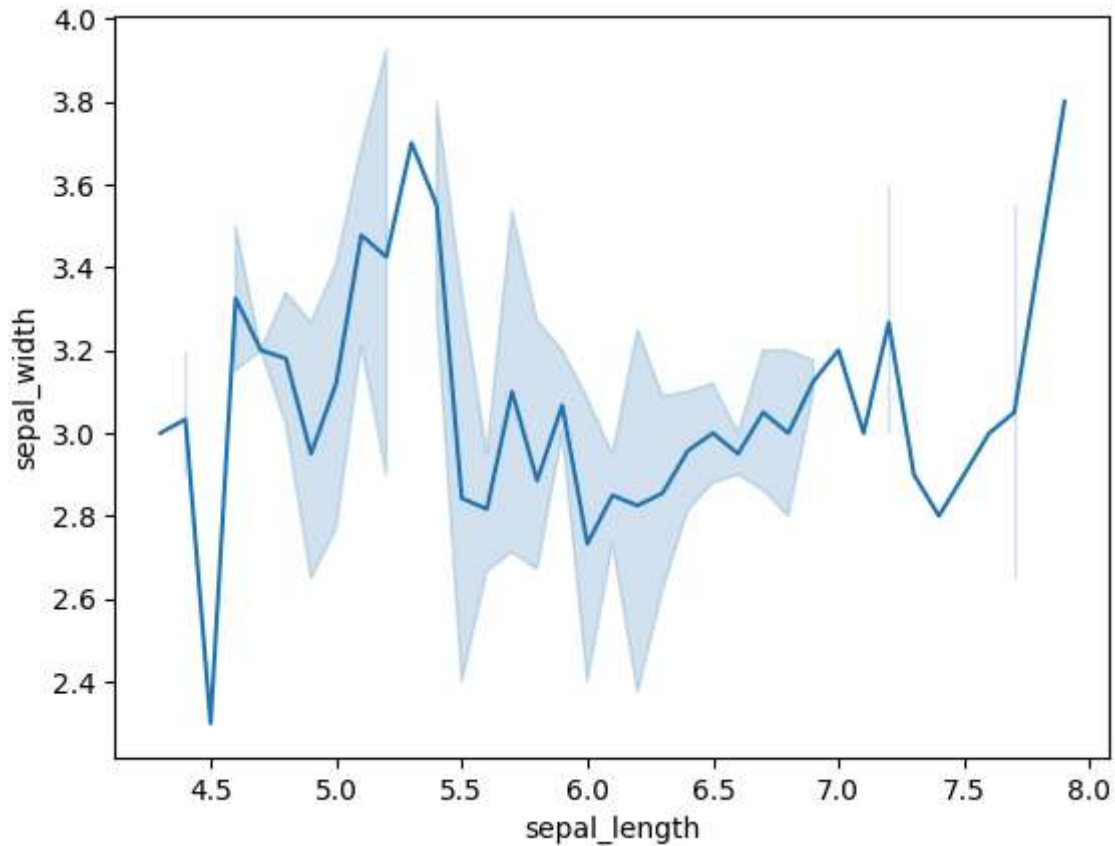
```
species
setosa      50
versicolor  50
virginica   50
dtype: int64
```

line plot

line plot is used to visualise the continuous variable , we can pass one or two variable based on our requirements ,y should be always numerical but x can be numerical or categorical

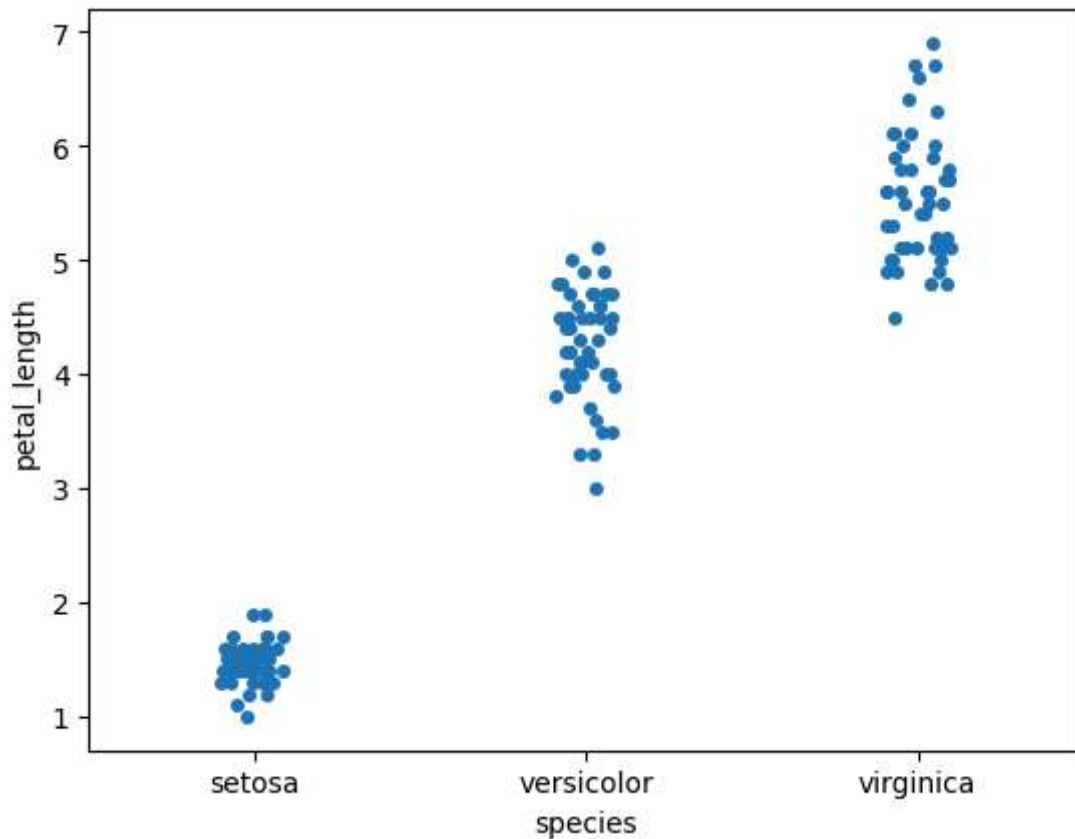
```
In [11]: sns.lineplot(x="sepal_length",y="sepal_width",data=iris)
```

```
Out[11]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



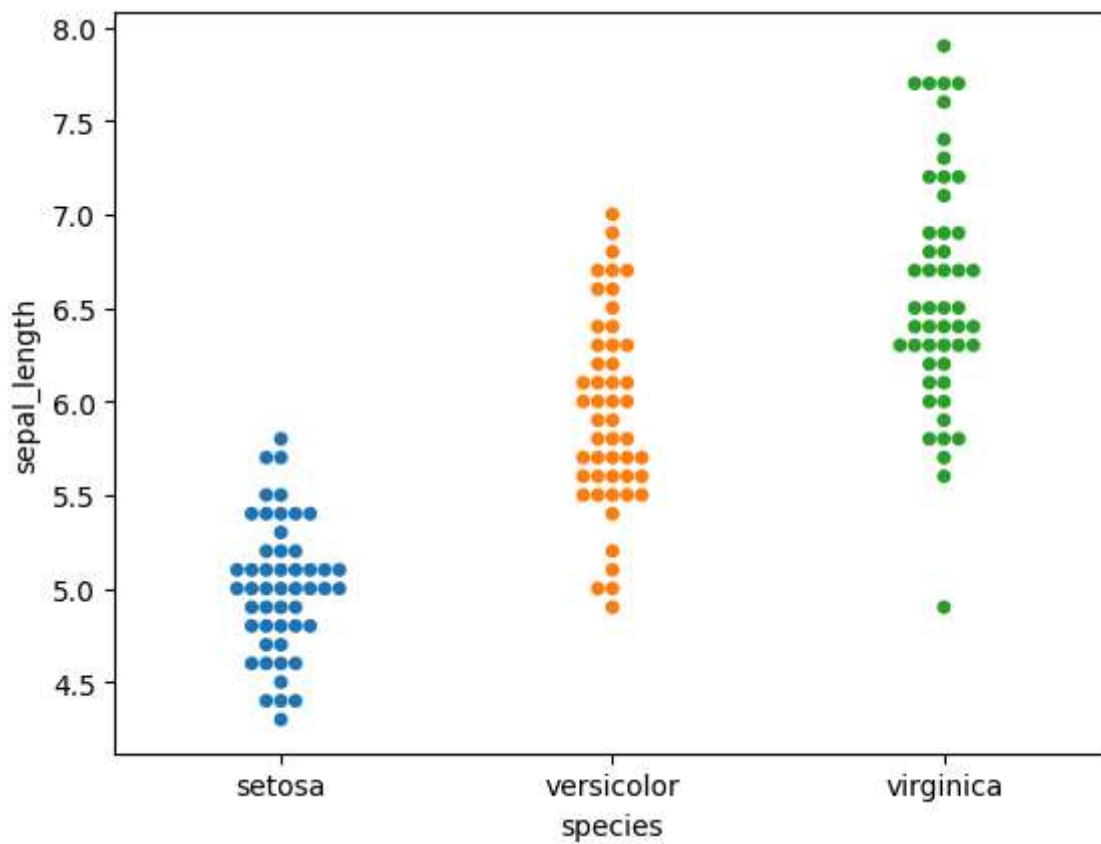
```
In [12]: sns.stripplot(x="species",y="petal_length",data=iris)
```

```
Out[12]: <Axes: xlabel='species', ylabel='petal_length'>
```



```
In [15]: sns.swarmplot(x="species",y="sepal_length",data=iris,hue="species")
```

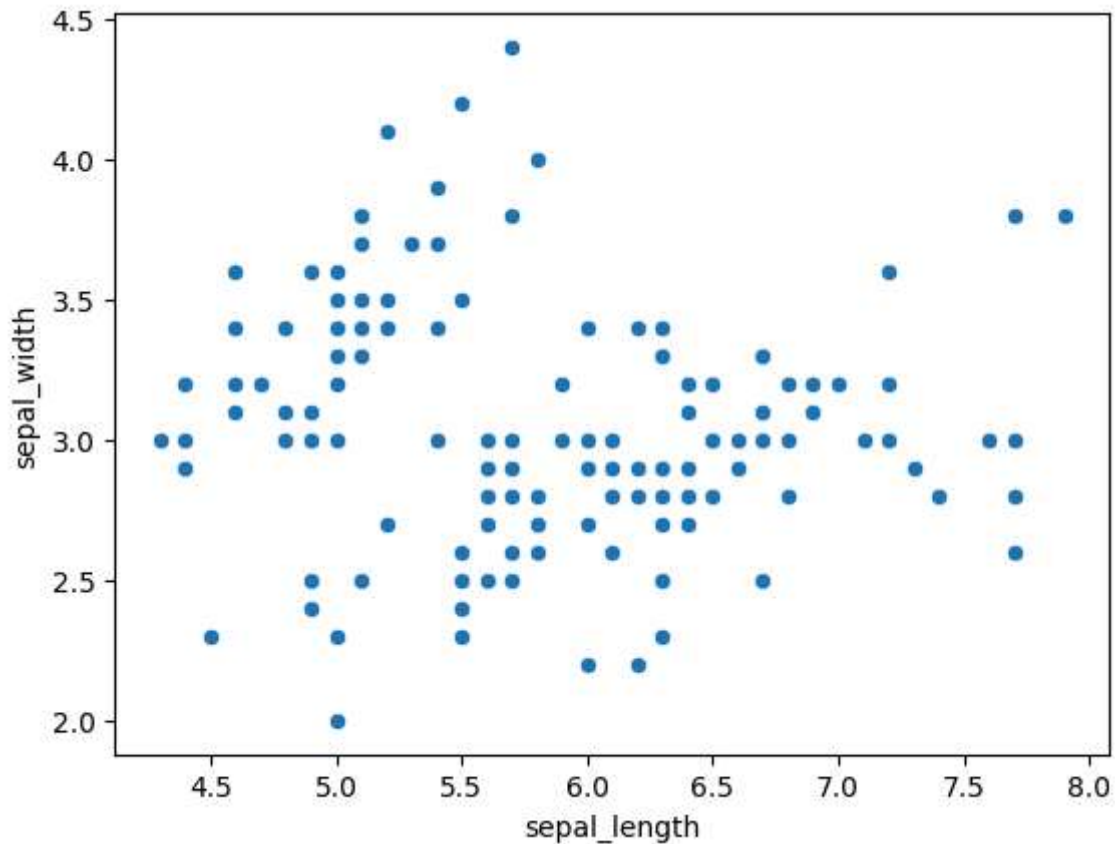
```
Out[15]: <Axes: xlabel='species', ylabel='sepal_length'>
```



A scatterplot shows the relationship between two numerical variables as individual data points (dots).

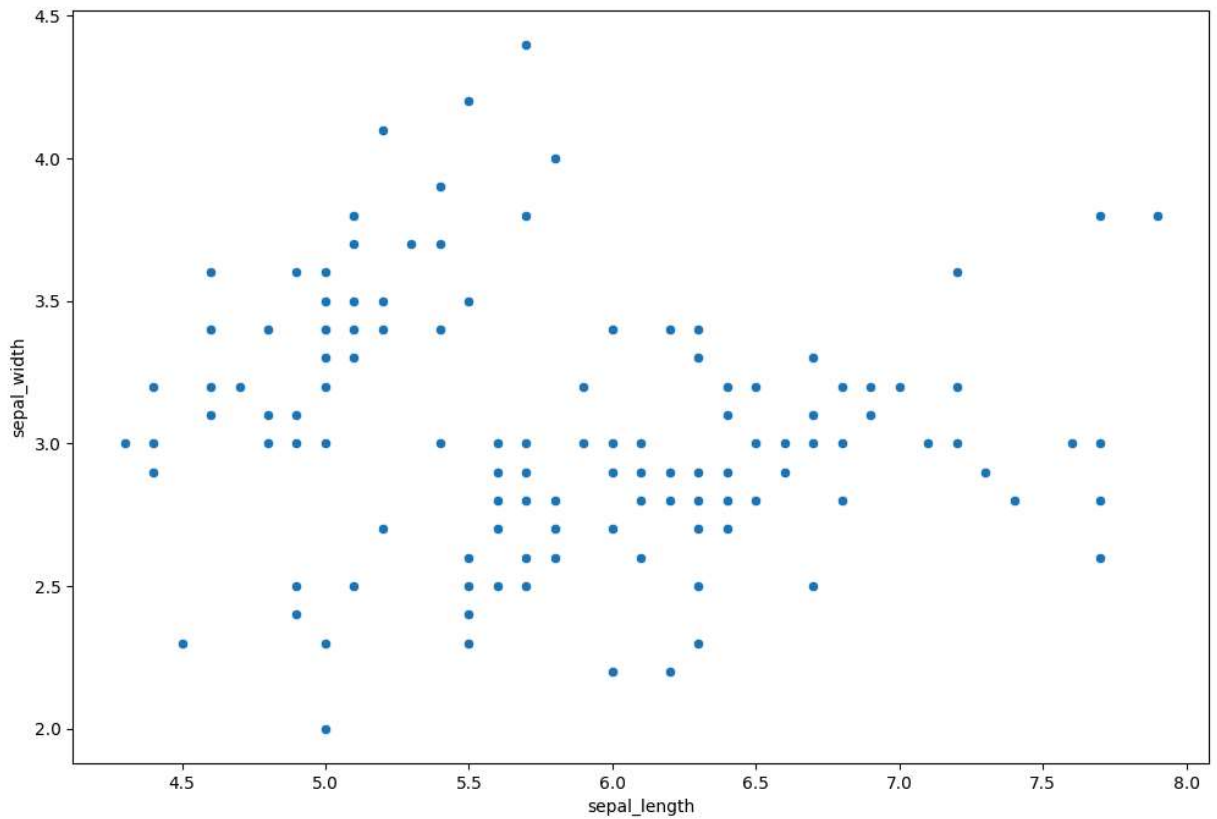
```
In [16]: sns.scatterplot(x='sepal_length',y='sepal_width',data=iris)
```

```
Out[16]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



```
In [17]: plt.figure(figsize=(12,8))
sns.scatterplot(x='sepal_length',y='sepal_width',data=iris)
```

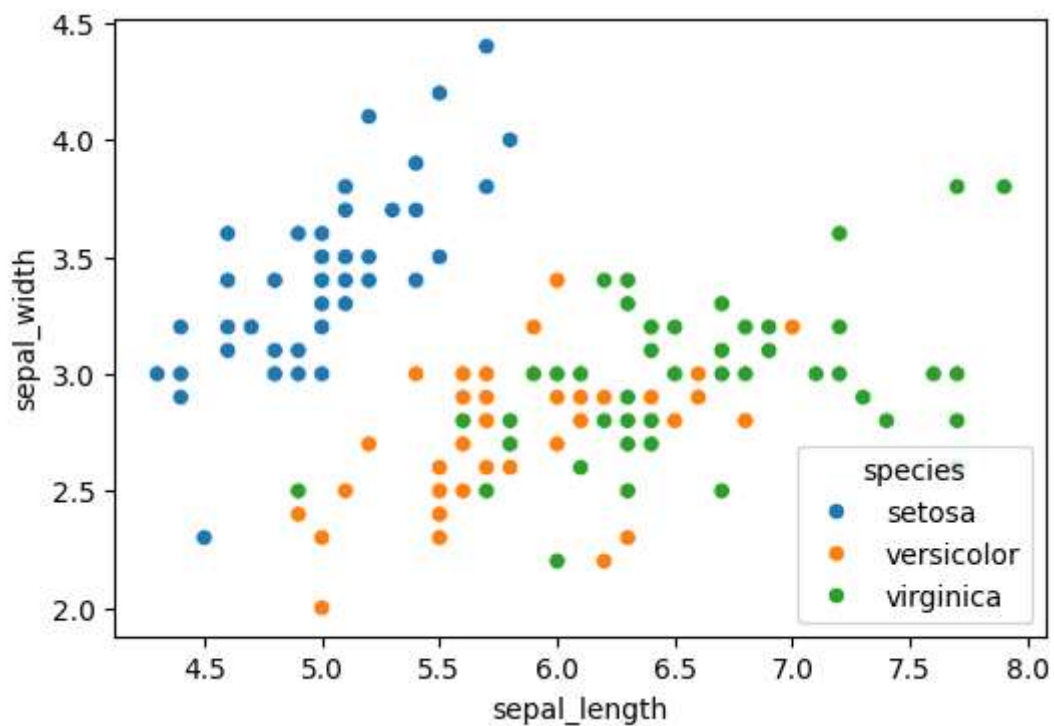
```
Out[17]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



scatterplot parameters

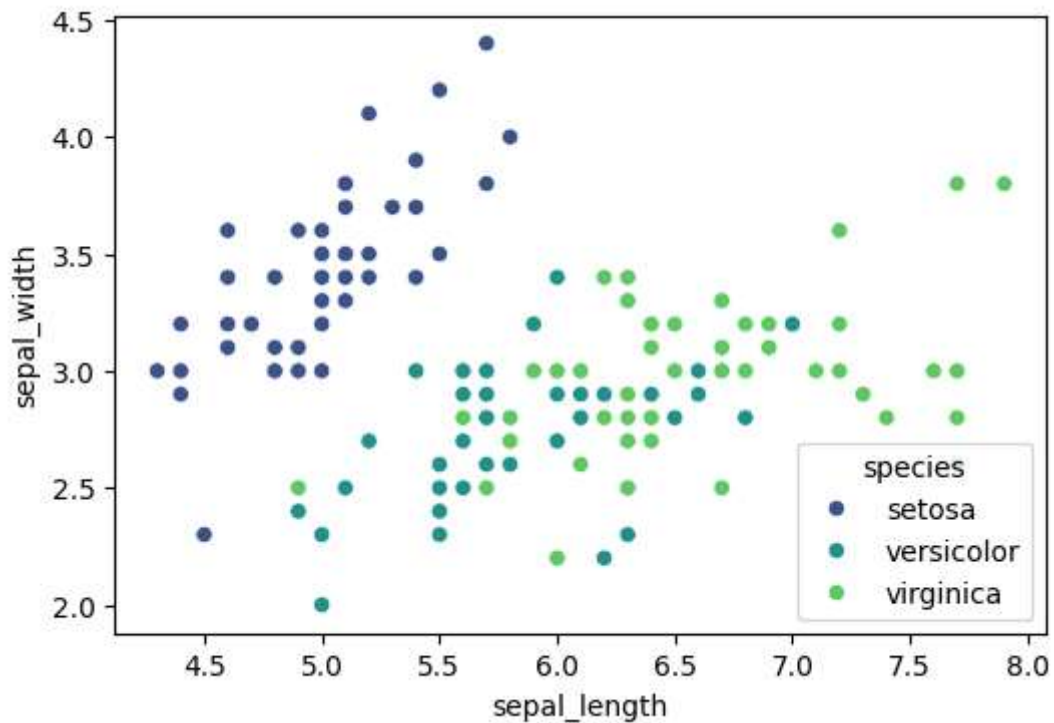
```
In [18]: plt.figure(figsize=(6,4))  
sns.scatterplot(x='sepal_length',y='sepal_width',data=iris,hue='species')
```

```
Out[18]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



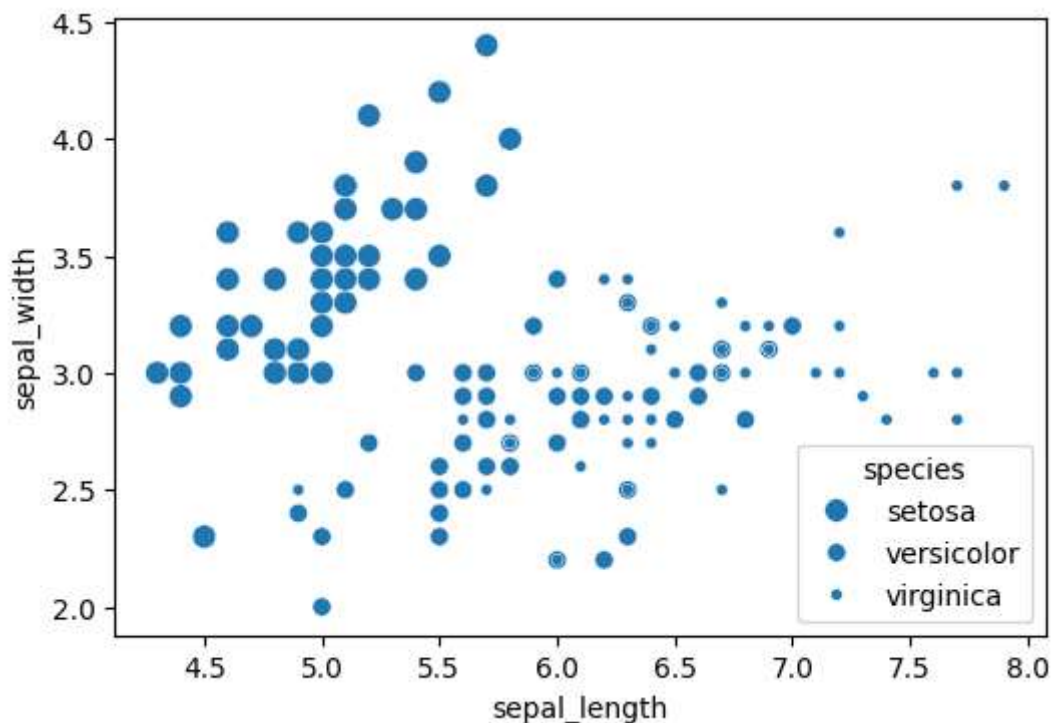
```
In [19]: plt.figure(figsize=(6,4))  
sns.scatterplot(x='sepal_length',y='sepal_width',data=iris,hue='species',palette='v
```

```
Out[19]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



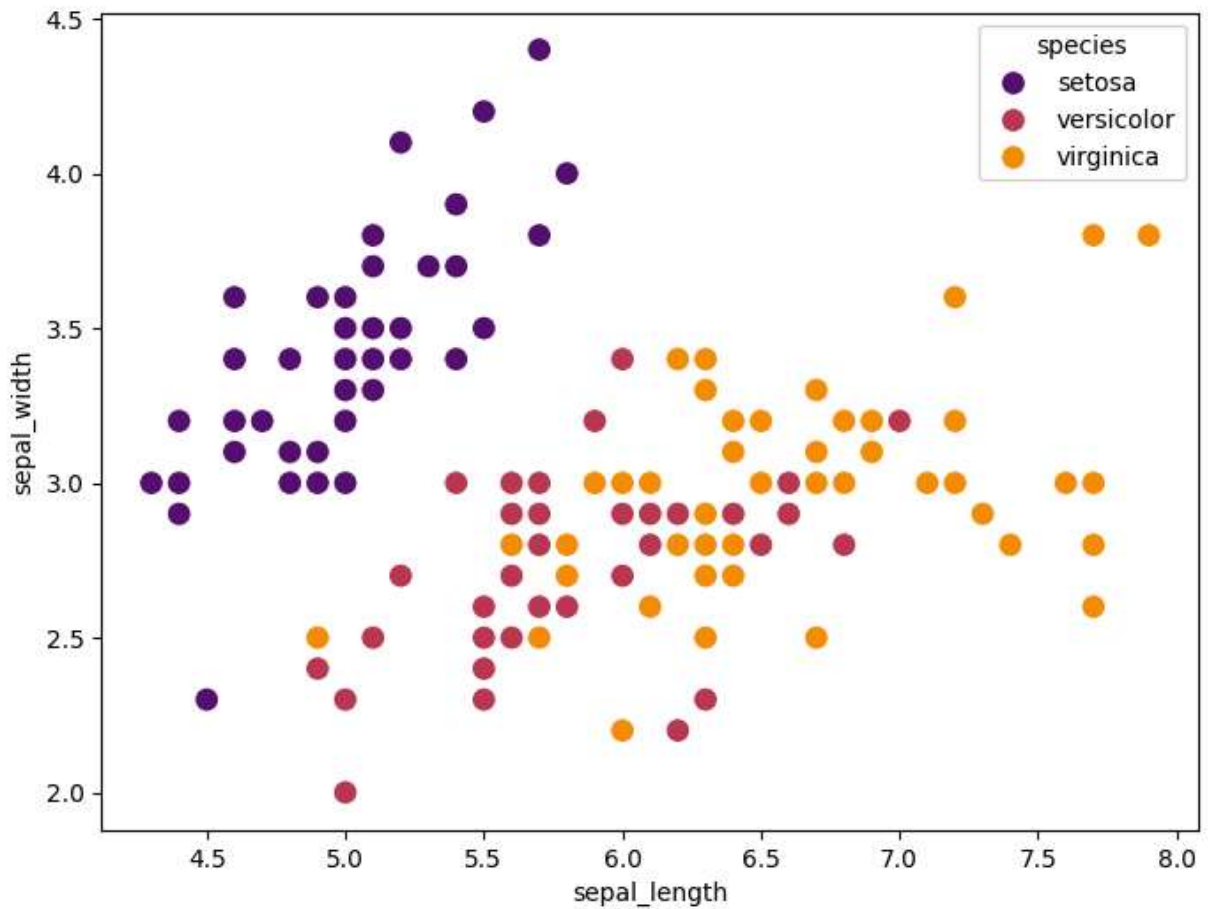
```
In [20]: plt.figure(figsize=(6,4))  
sns.scatterplot(x='sepal_length',y='sepal_width',data=iris,size='species')
```

```
Out[20]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



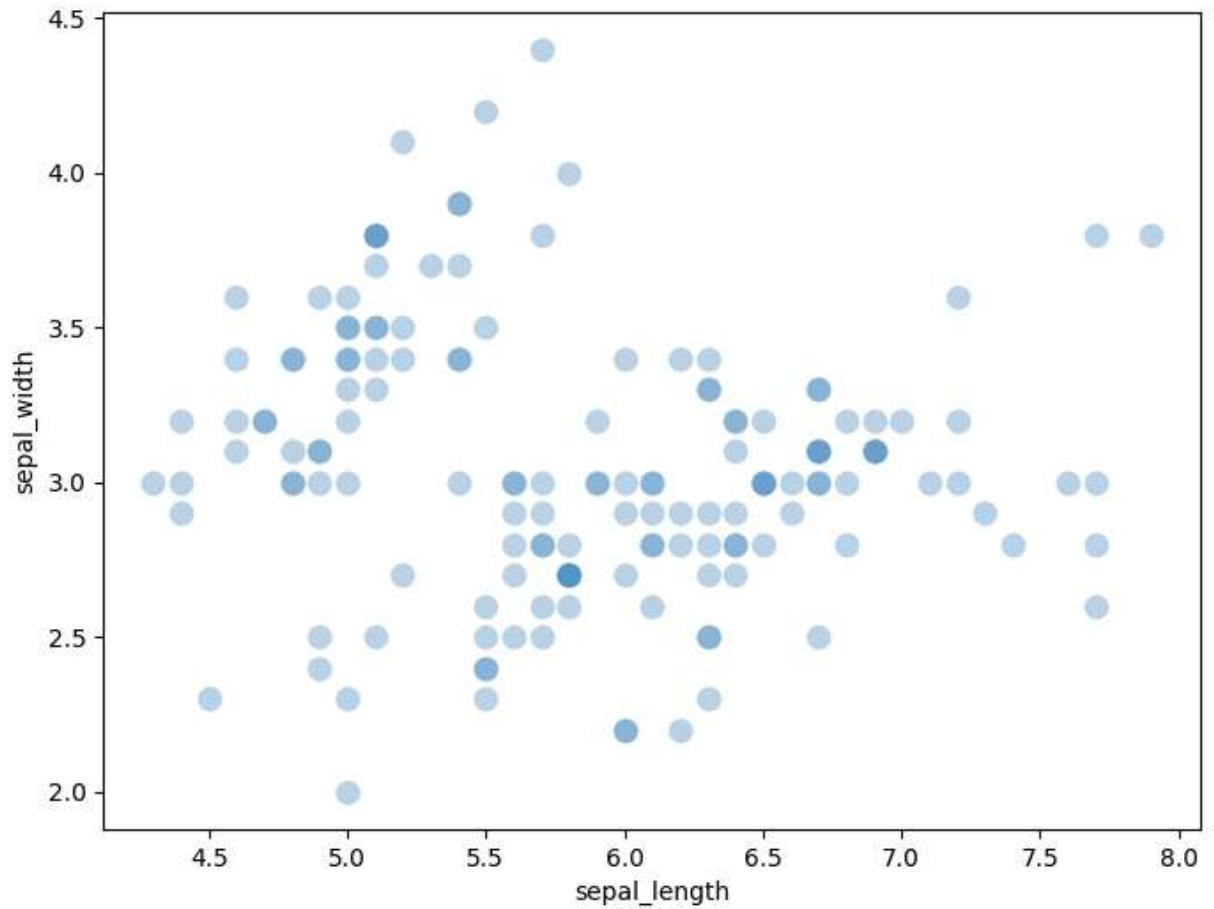
```
In [21]: plt.figure(figsize=(8,6))  
sns.scatterplot(x='sepal_length',y='sepal_width',data=iris,s=100,hue='species',pale
```

```
Out[21]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



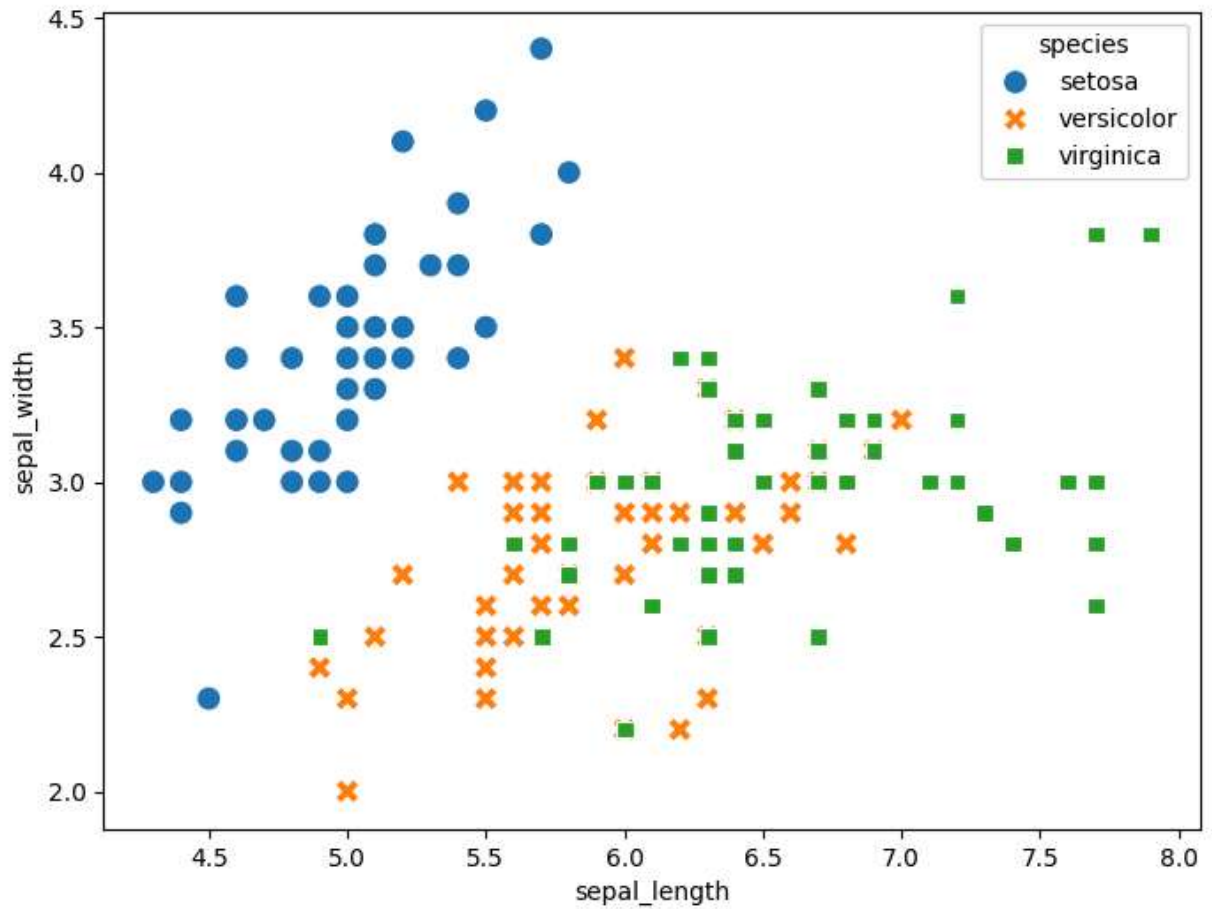
```
In [22]: plt.figure(figsize=(8,6))  
sns.scatterplot(x='sepal_length',y='sepal_width',data=iris,s=100,linewidth=0,alpha=
```

```
Out[22]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```

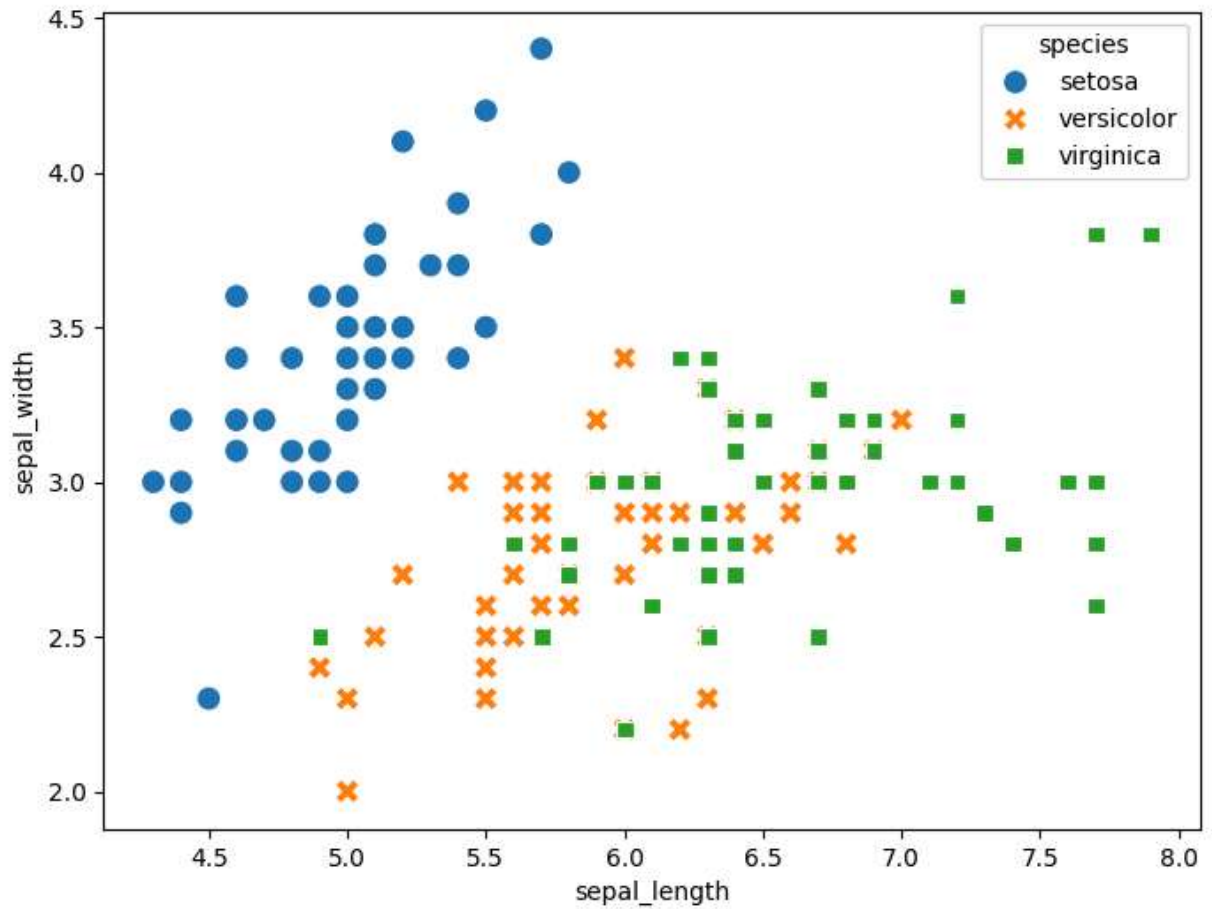
```
In [23]: plt.figure(figsize=(8,6))
sns.scatterplot(x='sepal_length',y='sepal_width',data=iris,style='species',hue='spe
```

```
Out[23]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```



exporting a seaborn figure

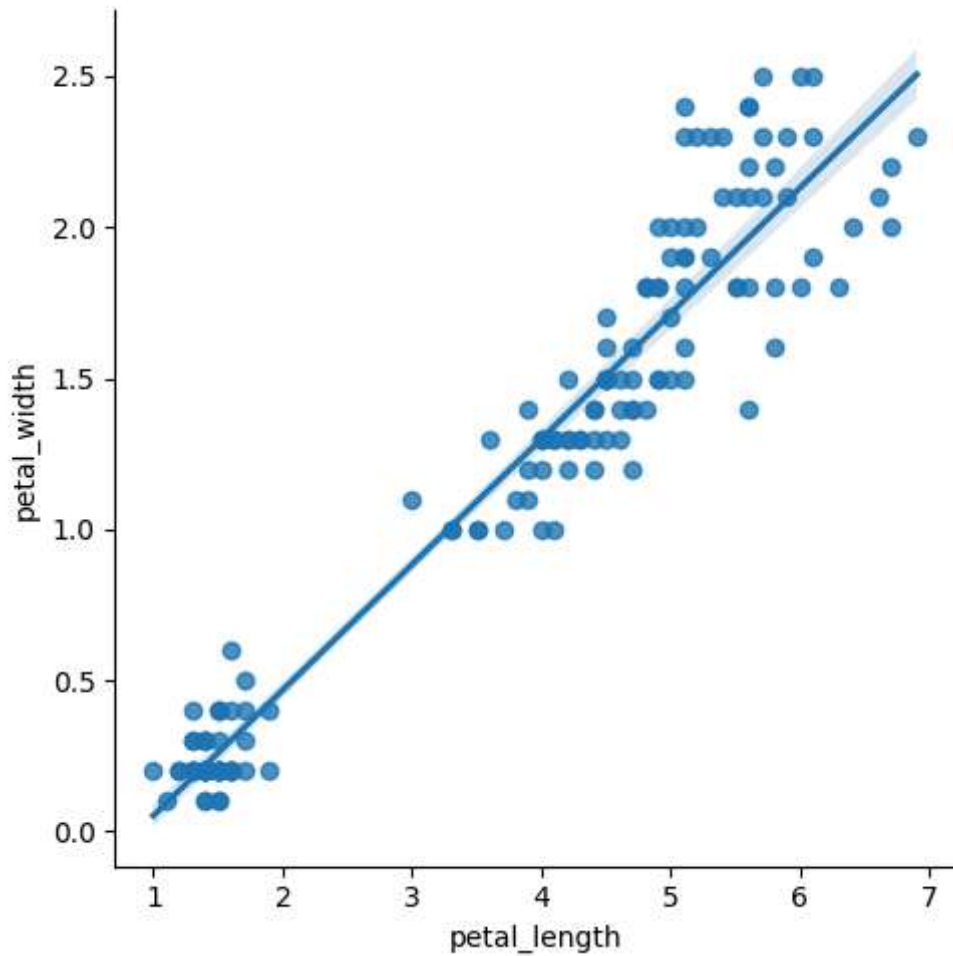
```
In [24]: plt.figure(figsize=(8,6))
sns.scatterplot(x='sepal_length',y='sepal_width',data=iris,style='species',hue='spe
plt.savefig('example_scatter.jpg')
```



Implot(): linear model plot ,it plots a scatter plot between two variables

```
In [25]: sns.lmplot(x='petal_length',y='petal_width',data=iris)
```

```
Out[25]: <seaborn.axisgrid.FacetGrid at 0x1c3abdf7b0>
```

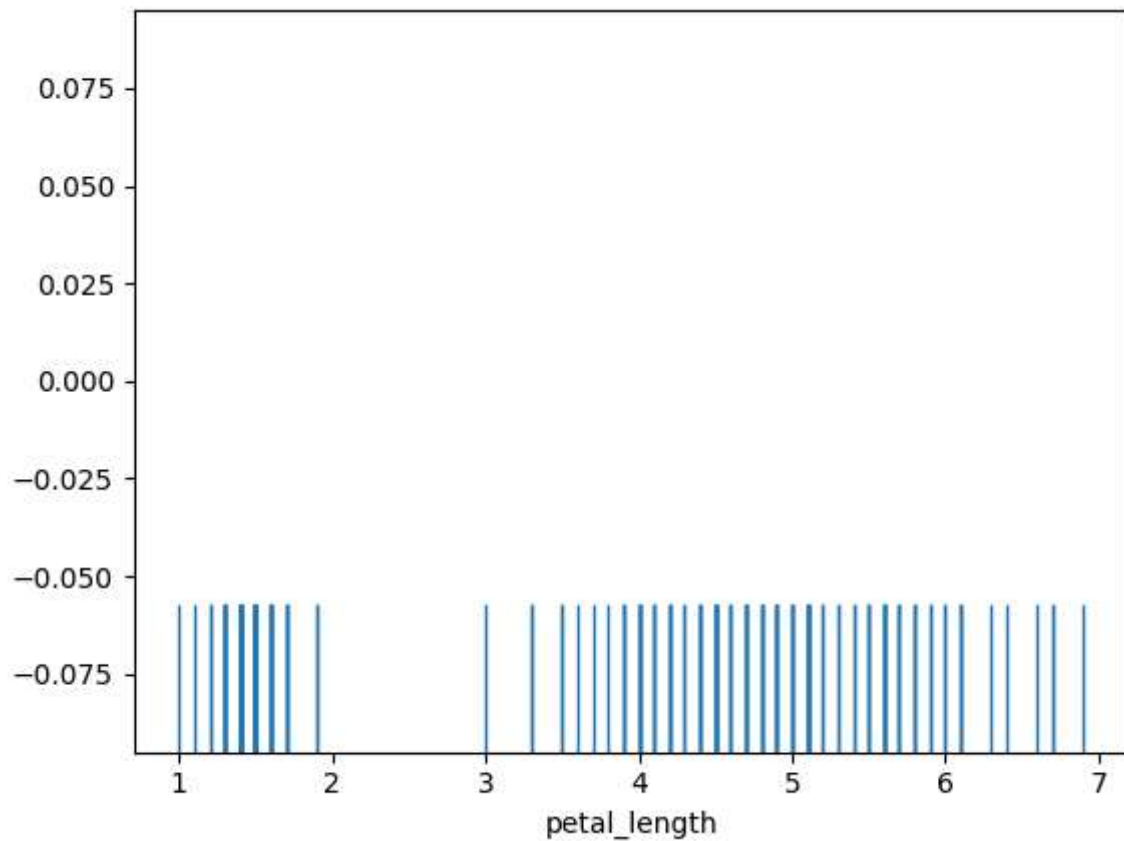


distribution plots

rugplot

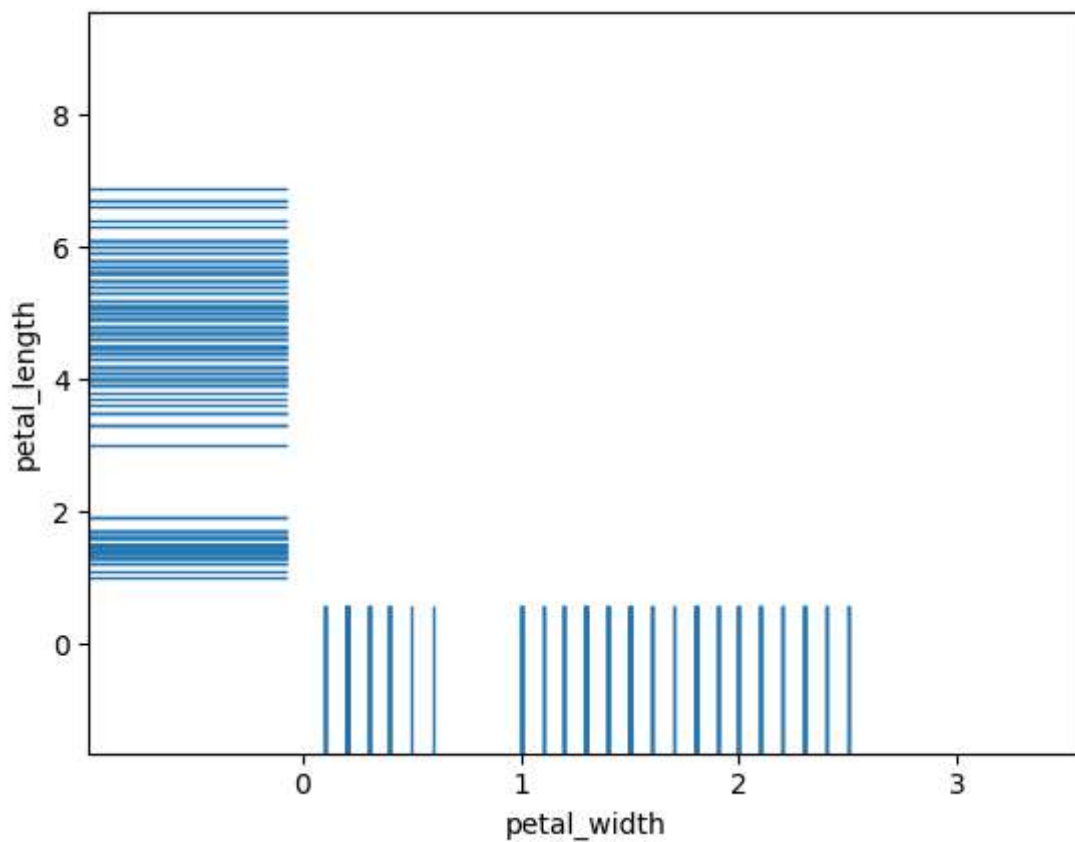
```
In [26]: sns.rugplot(x='petal_length',data=iris,height=0.2)
```

```
Out[26]: <Axes: xlabel='petal_length'>
```



```
In [27]: sns.rugplot(x='petal_width',y='petal_length',data=iris,height=0.2)
```

```
Out[27]: <Axes: xlabel='petal_width', ylabel='petal_length'>
```

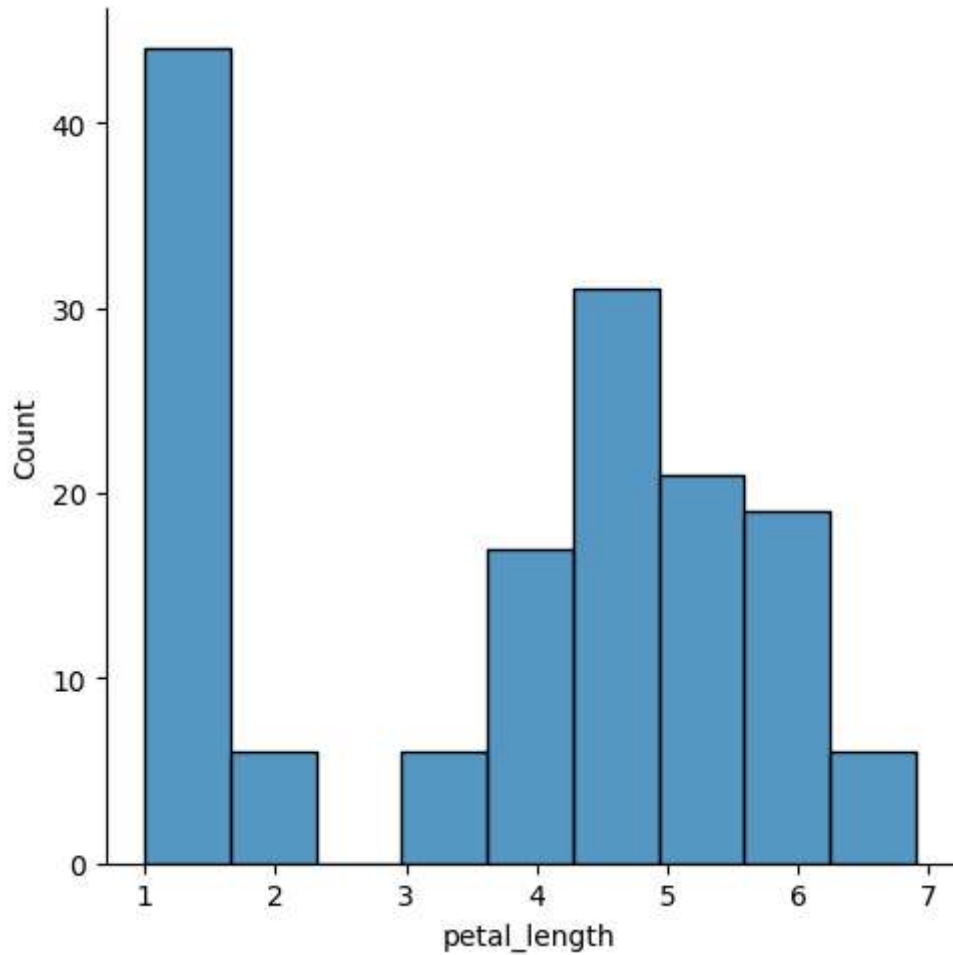


displot and histplot

displot is designed for visualising univariate distribution

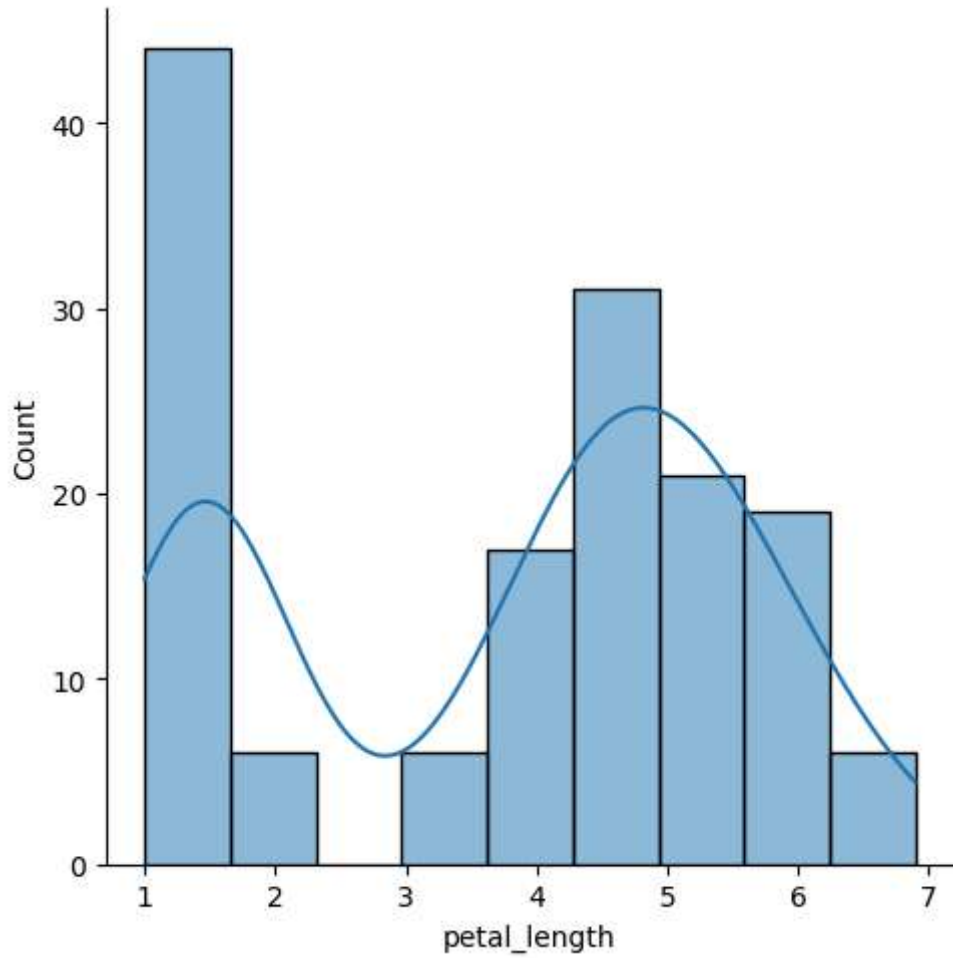
```
In [28]: sns.displot(data=iris,x='petal_length')
```

```
Out[28]: <seaborn.axisgrid.FacetGrid at 0x1c3b01dc910>
```



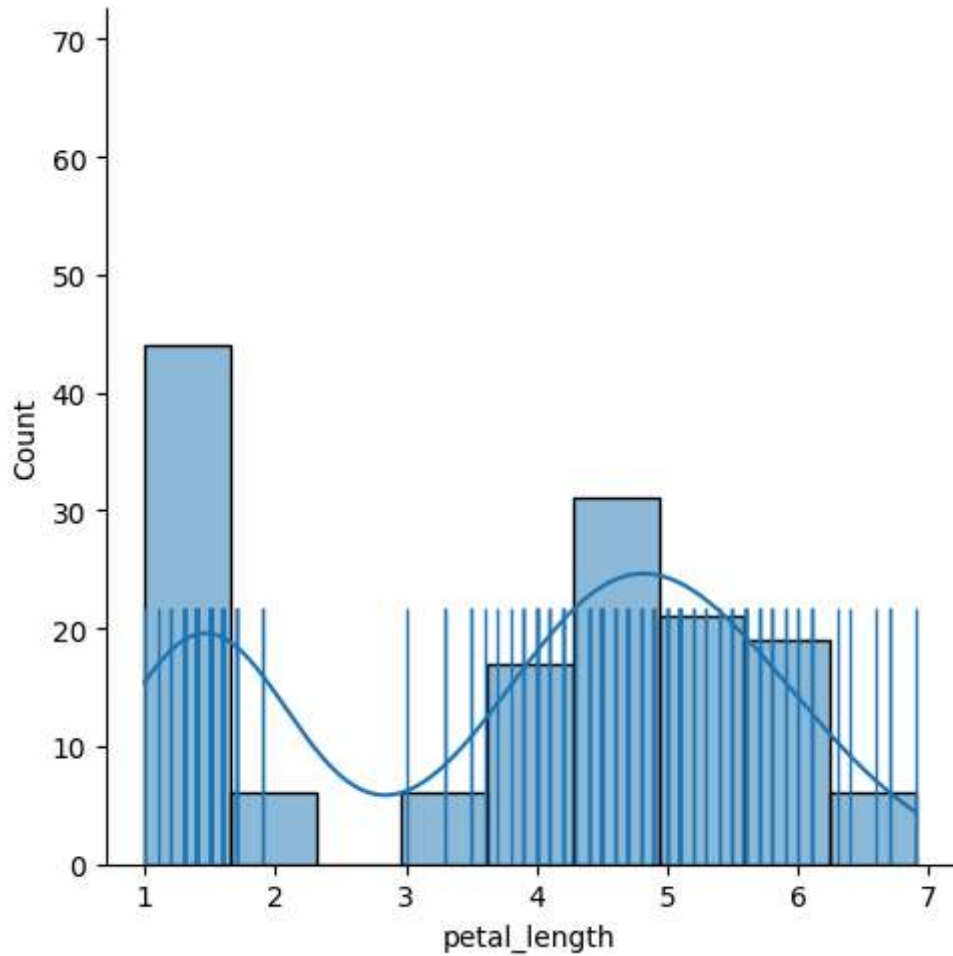
```
In [29]: sns.displot(data=iris,x='petal_length',kde=True)
```

```
Out[29]: <seaborn.axisgrid.FacetGrid at 0x1c3b022ead0>
```



```
In [30]: sns.displot(x='petal_length',data=iris,kde=True,rug=True ,rug_kws= {"height":0.3})
```

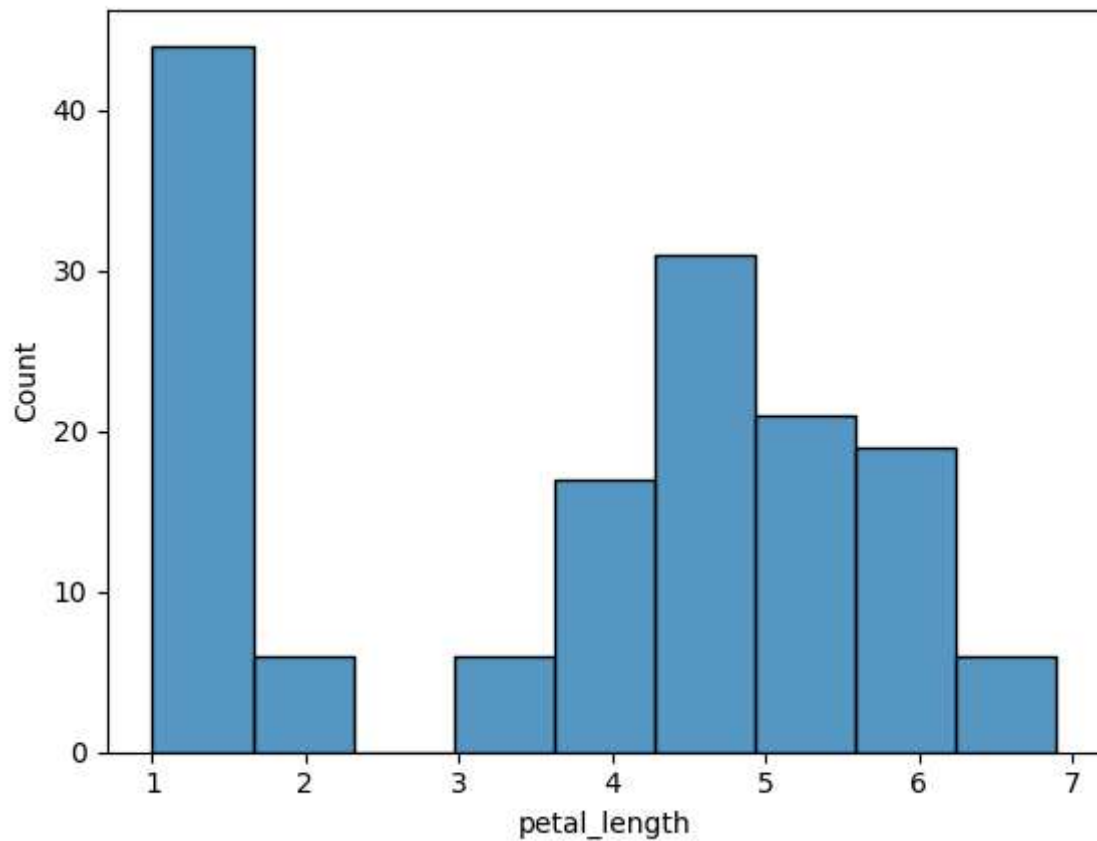
```
Out[30]: <seaborn.axisgrid.FacetGrid at 0x1c3b02a2ad0>
```



histplot is used to visualise single variable, we pass several arguments that are (data, x, y, bins, kde=true or false)

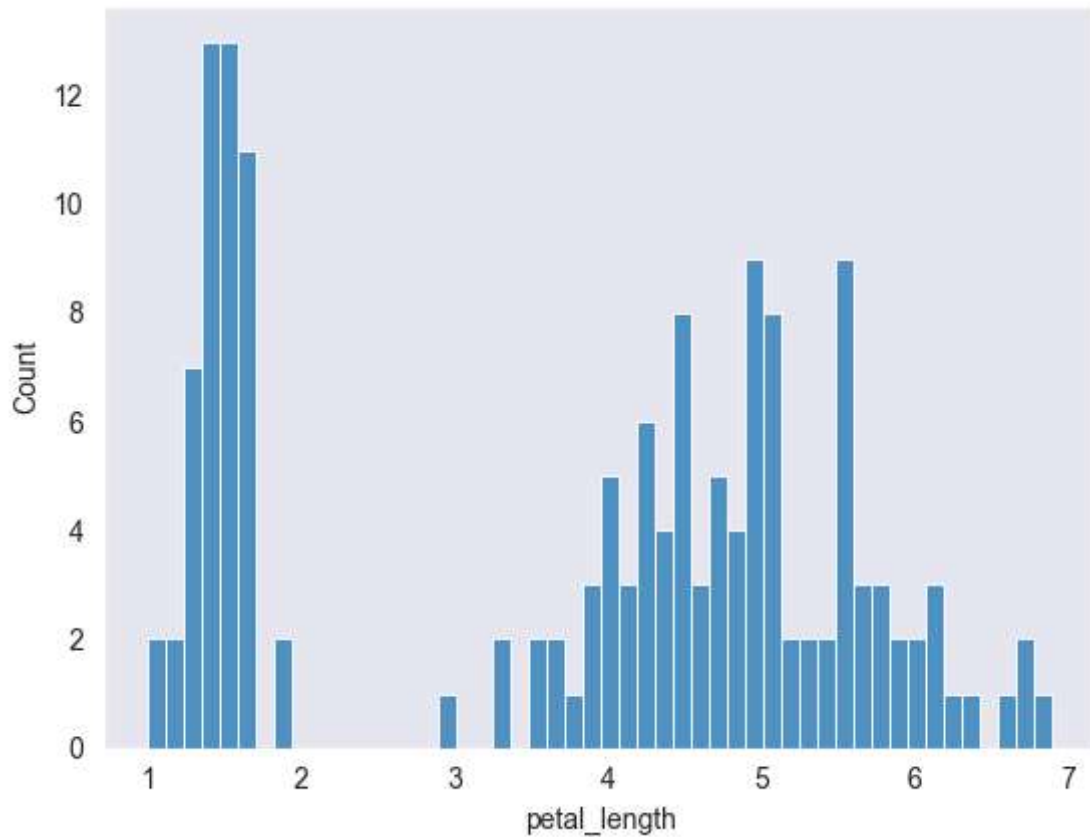
```
In [31]: sns.histplot(data=iris, x='petal_length')
```

```
Out[31]: <Axes: xlabel='petal_length', ylabel='Count'>
```

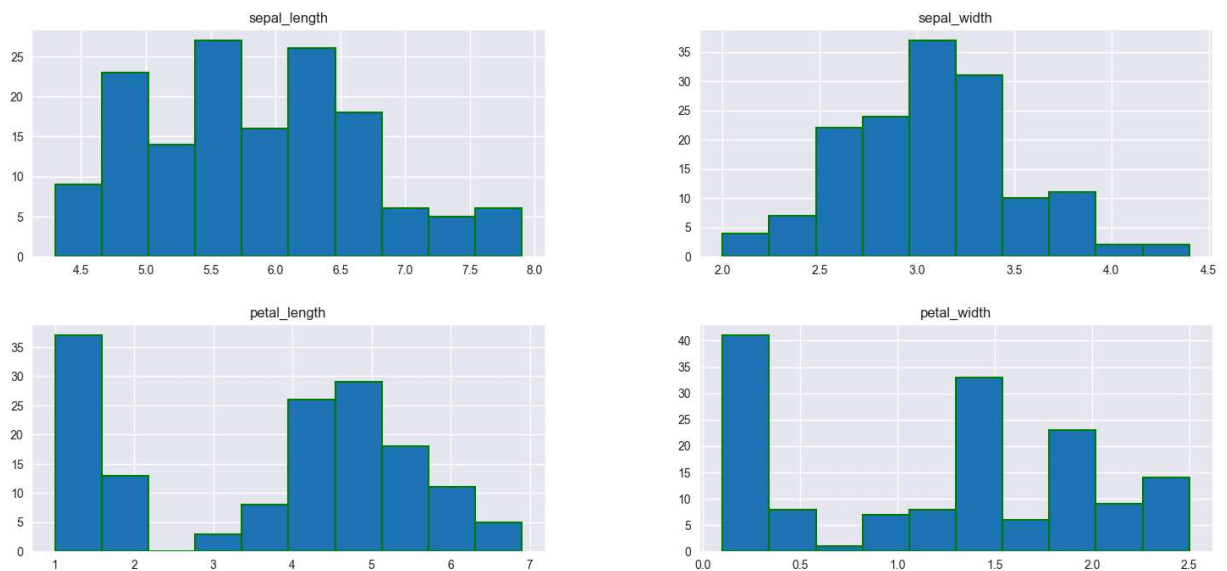



```
In [32]: sns.set_style('dark')
sns.histplot(data=iris, x='petal_length', bins=50)
```

```
Out[32]: <Axes: xlabel='petal_length', ylabel='Count'>
```



```
In [33]: iris.hist(edgecolor='green',linewidth=1.5,figsize=(18,8));  
plt.show()
```



categorical plot

boxplot

count plot

Bar plot

violin plot

box plot is a categorical plot x axis should be in categorical and y axis should be in numerical

Minimum value → lower whisker (not Q1)

25th percentile → Q1

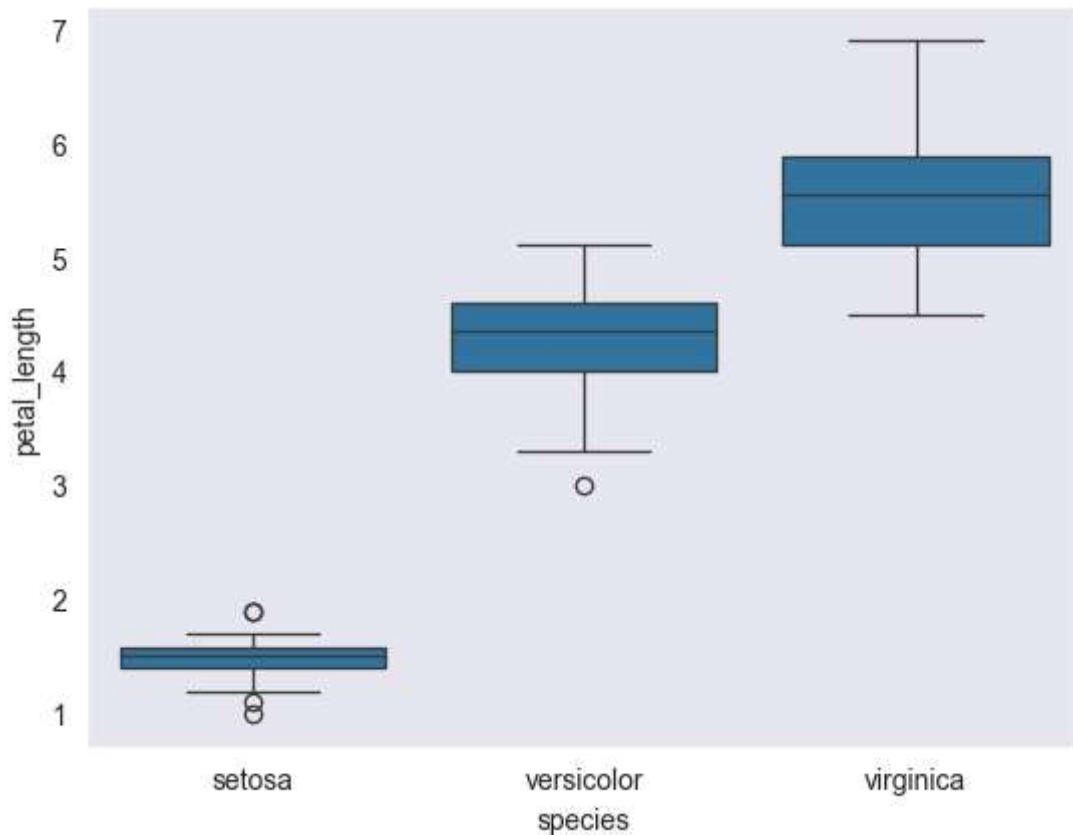
50th percentile → Median (Q2)

75th percentile → Q3

Maximum value → upper whisker (not Q3)

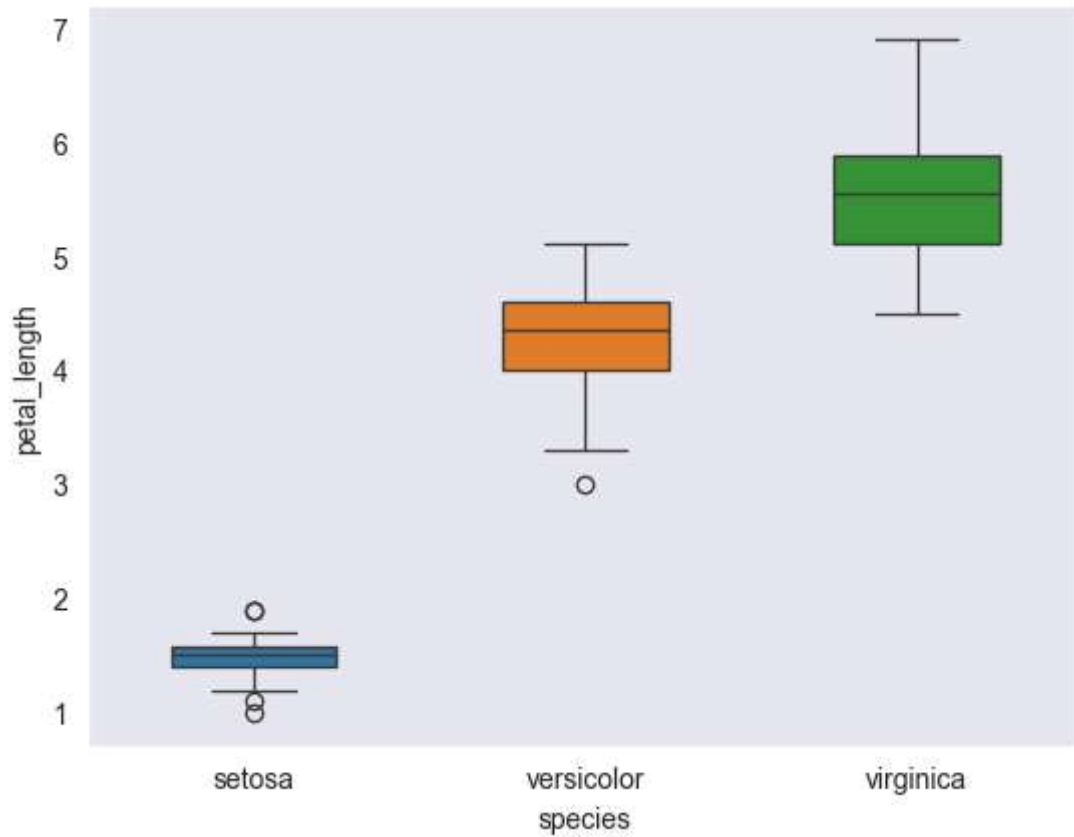
```
In [34]: sns.boxplot(y='petal_length',x='species',data=iris)
```

```
Out[34]: <Axes: xlabel='species', ylabel='petal_length'>
```



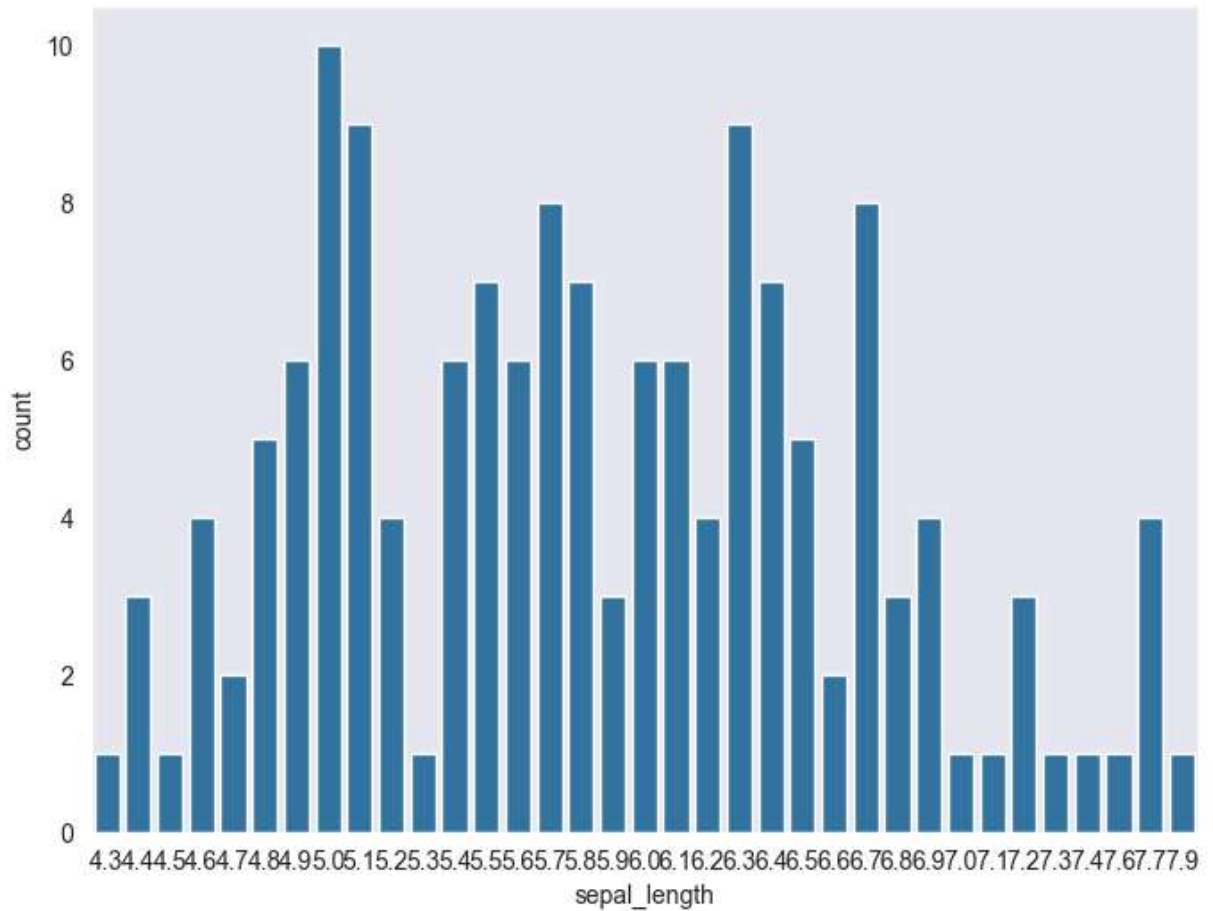
```
In [35]: sns.boxplot(y='petal_length',x='species',data=iris,hue='species',width=0.5)
```

```
Out[35]: <Axes: xlabel='species', ylabel='petal_length'>
```



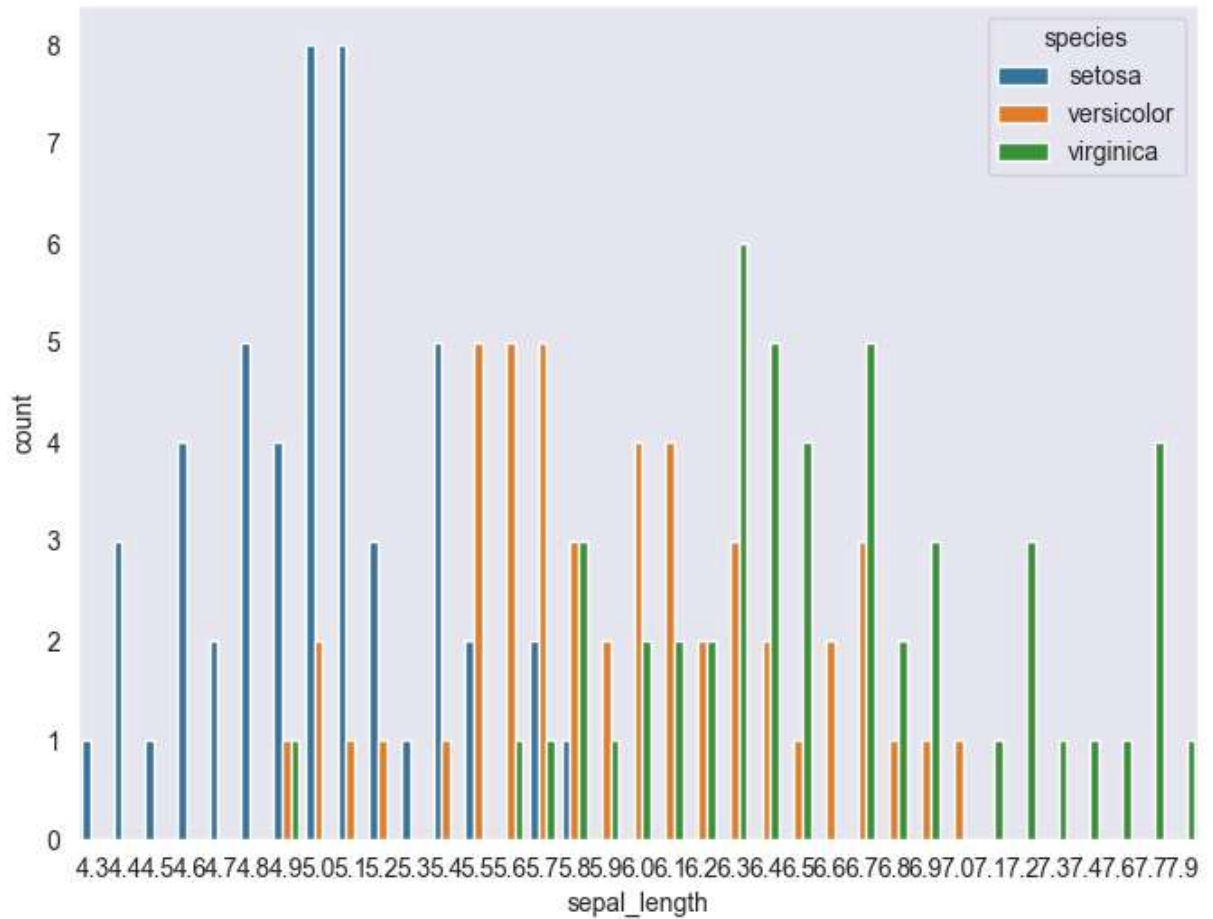
```
In [36]: plt.figure(figsize=(8,6),dpi=100)    # dpi :quality  
sns.countplot(x='sepal_length',data=iris)
```

```
Out[36]: <Axes: xlabel='sepal_length', ylabel='count'>
```



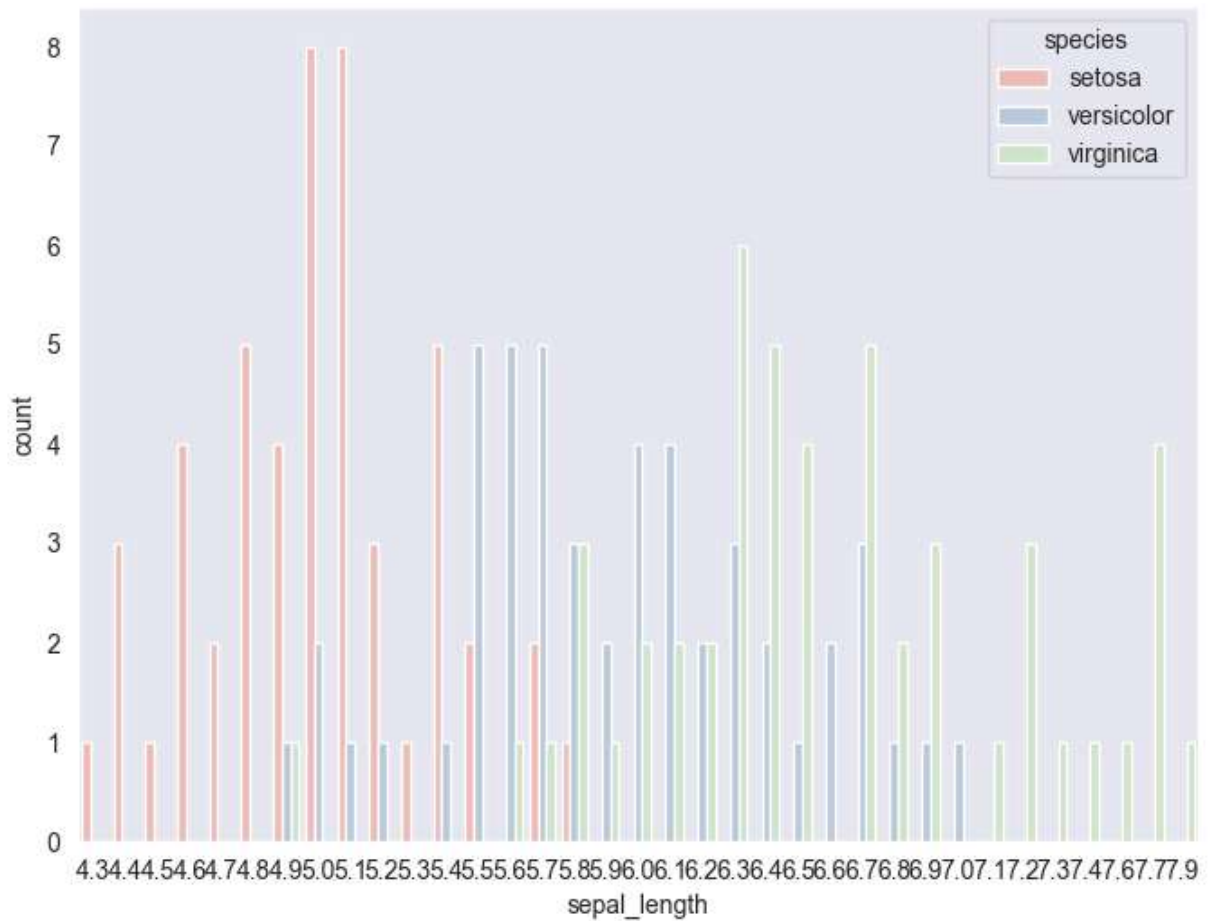
```
In [37]: plt.figure(figsize=(8,6),dpi=100) # dpi :quality
sns.countplot(x='sepal_length',data=iris,hue="species")
```

```
Out[37]: <Axes: xlabel='sepal_length', ylabel='count'>
```



```
In [38]: plt.figure(figsize=(8,6),dpi=100)    # dpi :quality  
sns.countplot(x='sepal_length',data=iris,hue="species",palette='Pastel1')
```

```
Out[38]: <Axes: xlabel='sepal_length', ylabel='count'>
```



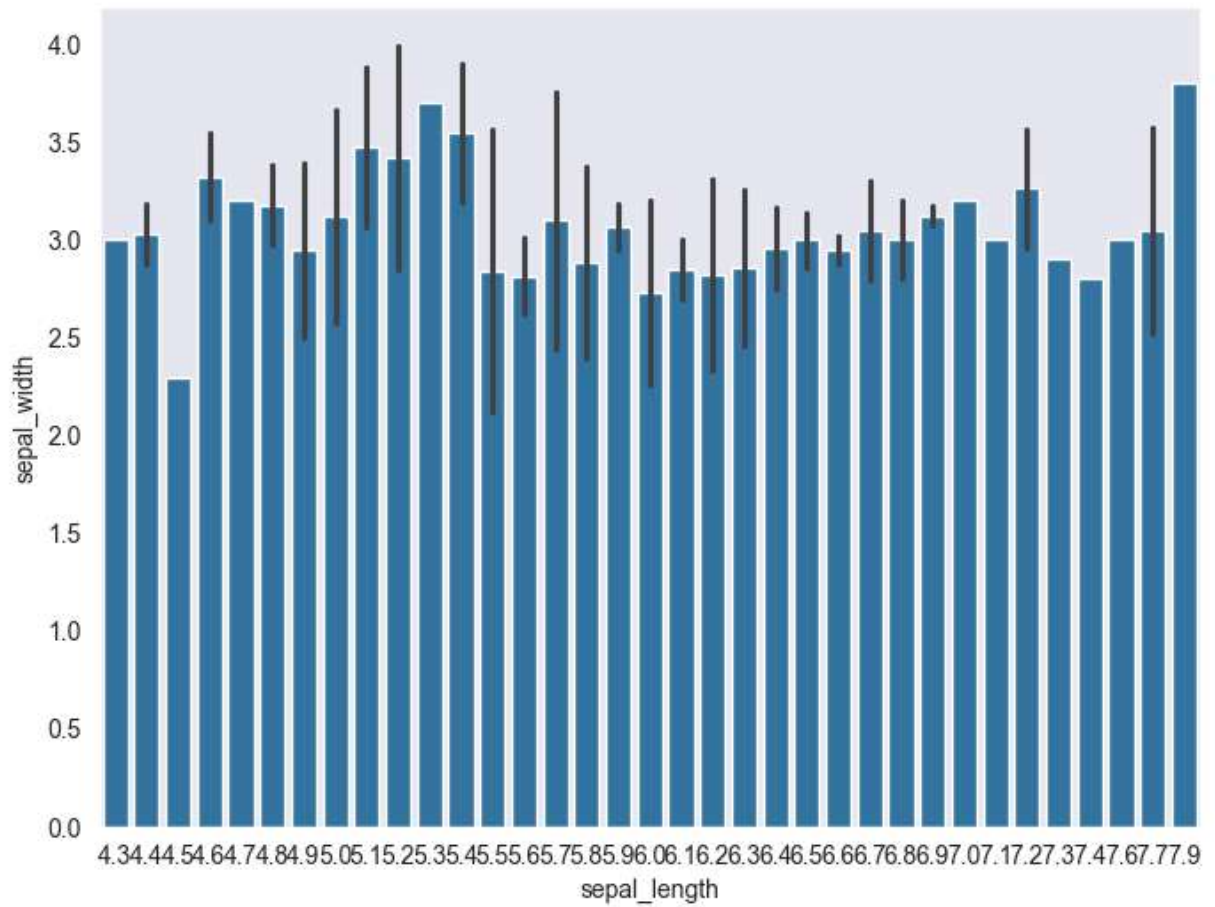
bar plot is categorical plot in seaborn

X-axis → should be categorical

Y-axis → should be numerical

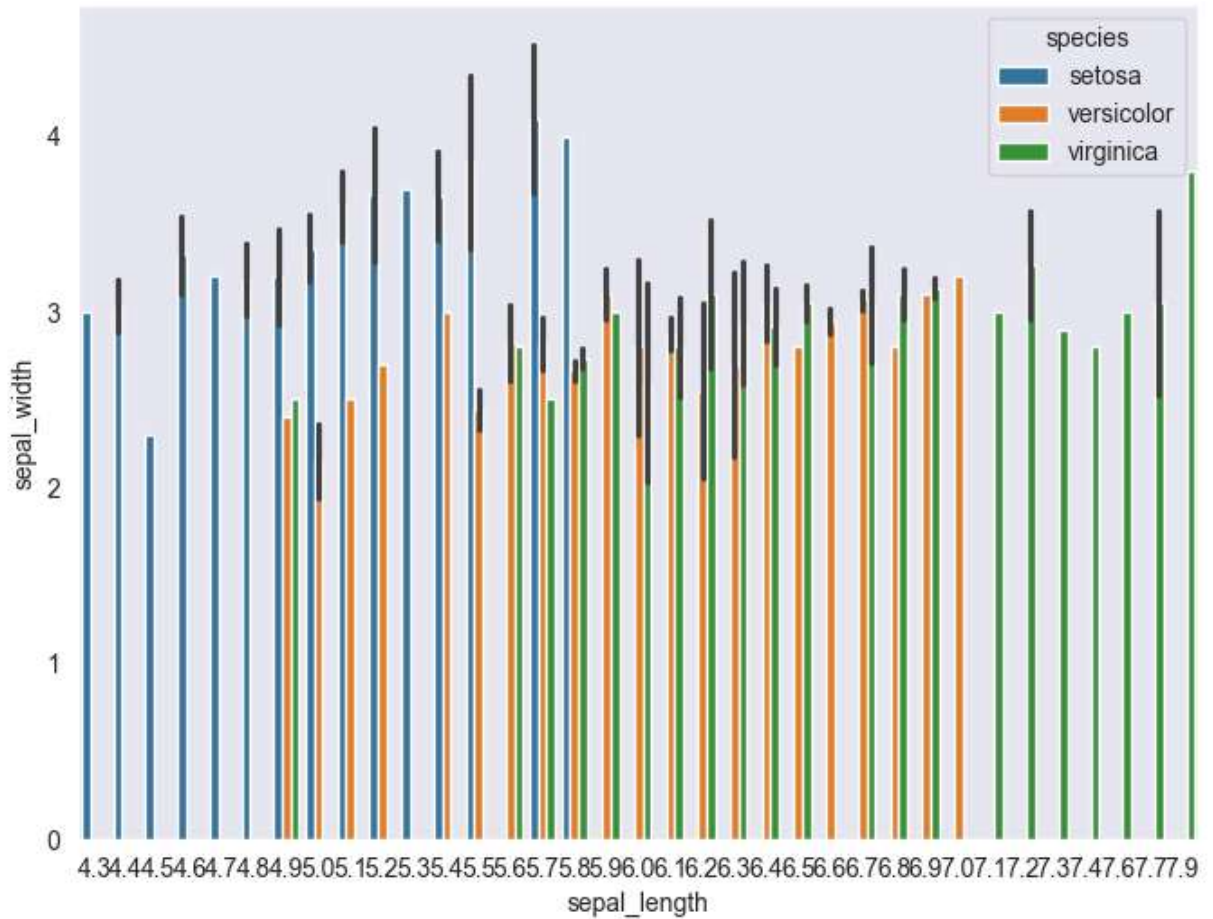
```
In [39]: plt.figure(figsize=(8,6),dpi=100)
sns.barplot(x='sepal_length',y='sepal_width',data=iris,estimator=np.mean,errorbar='')
```

```
Out[39]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```

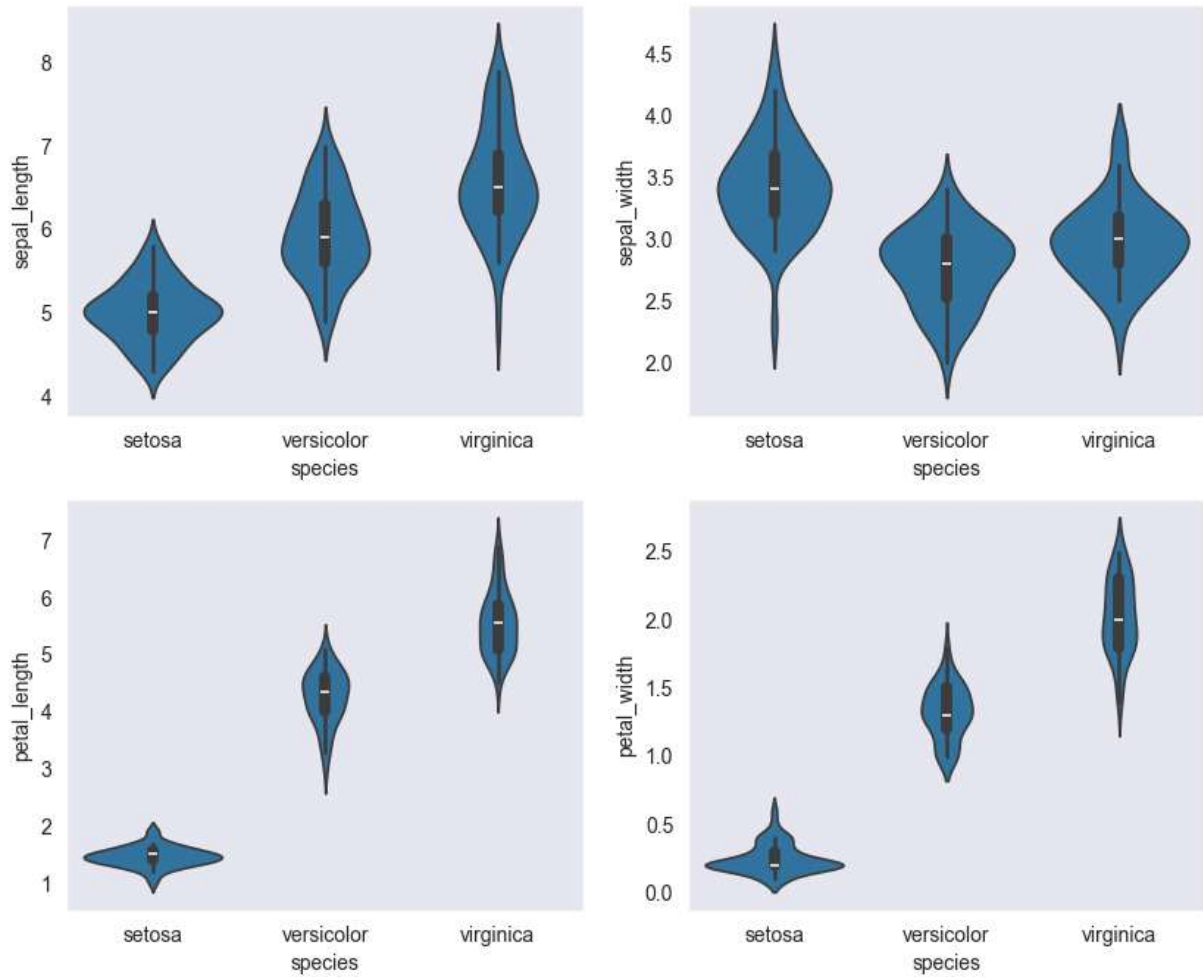


```
In [40]: plt.figure(figsize=(8,6),dpi=100)
sns.barplot(x='sepal_length',y='sepal_width',data=iris,estimator=np.mean,errorbar='')
```

```
Out[40]: <Axes: xlabel='sepal_length', ylabel='sepal_width'>
```

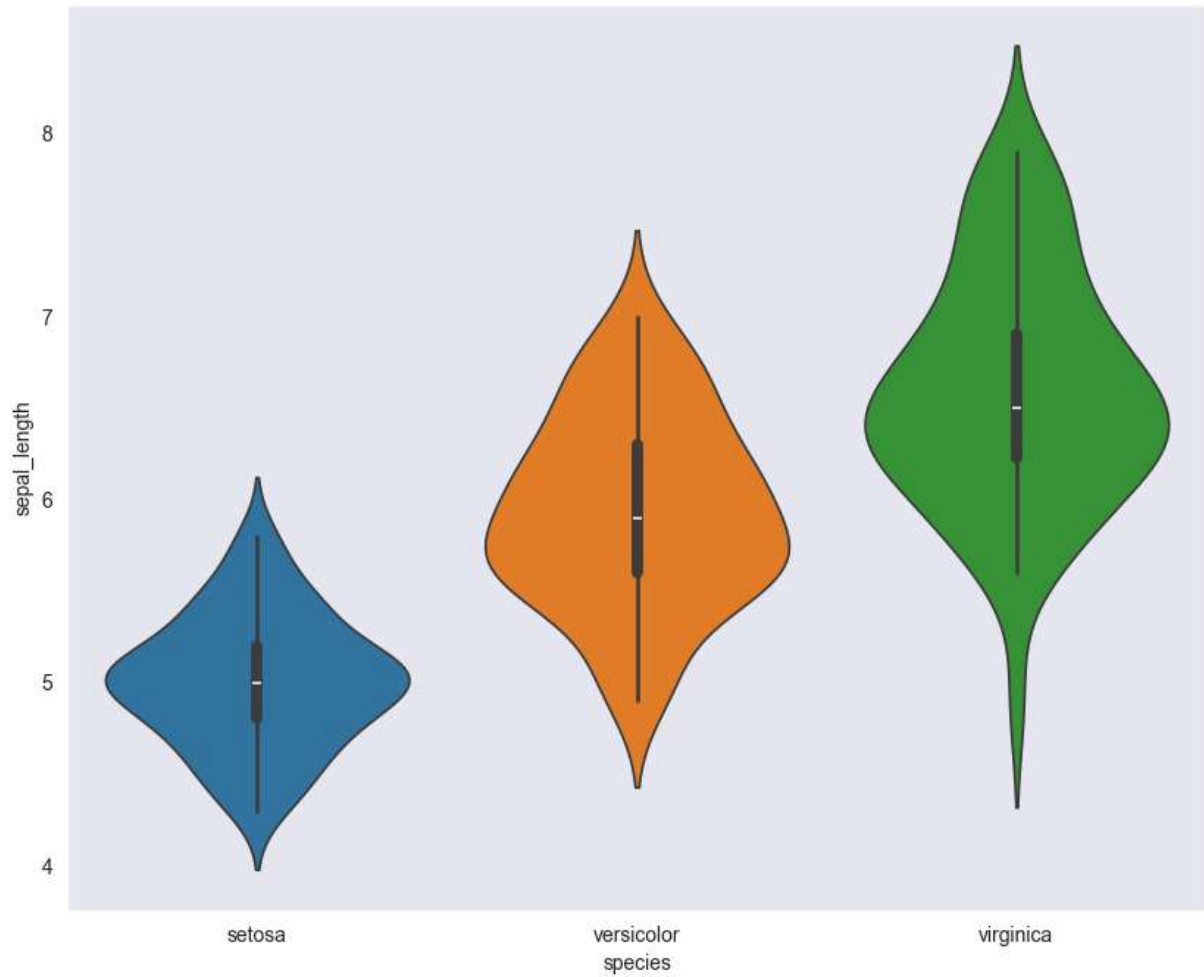



```
In [41]: plt.figure(figsize=(10,8));
plt.subplot(2,2,1)
sns.violinplot(x='species',y='sepal_length',data=iris)
plt.subplot(2,2,2)
sns.violinplot(x='species',y='sepal_width',data=iris)
plt.subplot(2,2,3)
sns.violinplot(x='species',y='petal_length',data=iris)
plt.subplot(2,2,4)
sns.violinplot(x='species',y='petal_width',data=iris);
```



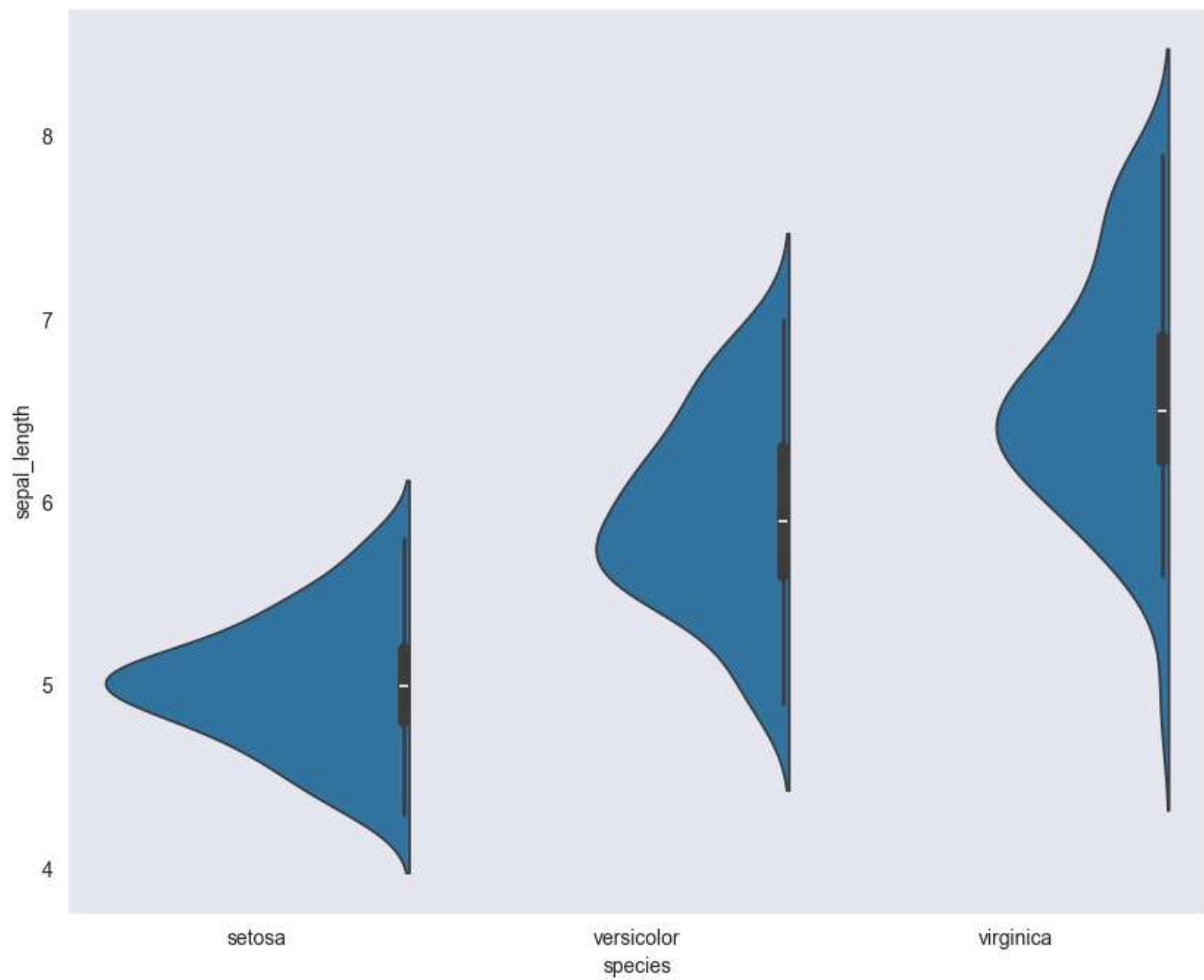
```
In [42]: plt.figure(figsize=(10,8));  
sns.violinplot(x='species',y='sepal_length',data=iris,hue='species')
```

```
Out[42]: <Axes: xlabel='species', ylabel='sepal_length'>
```



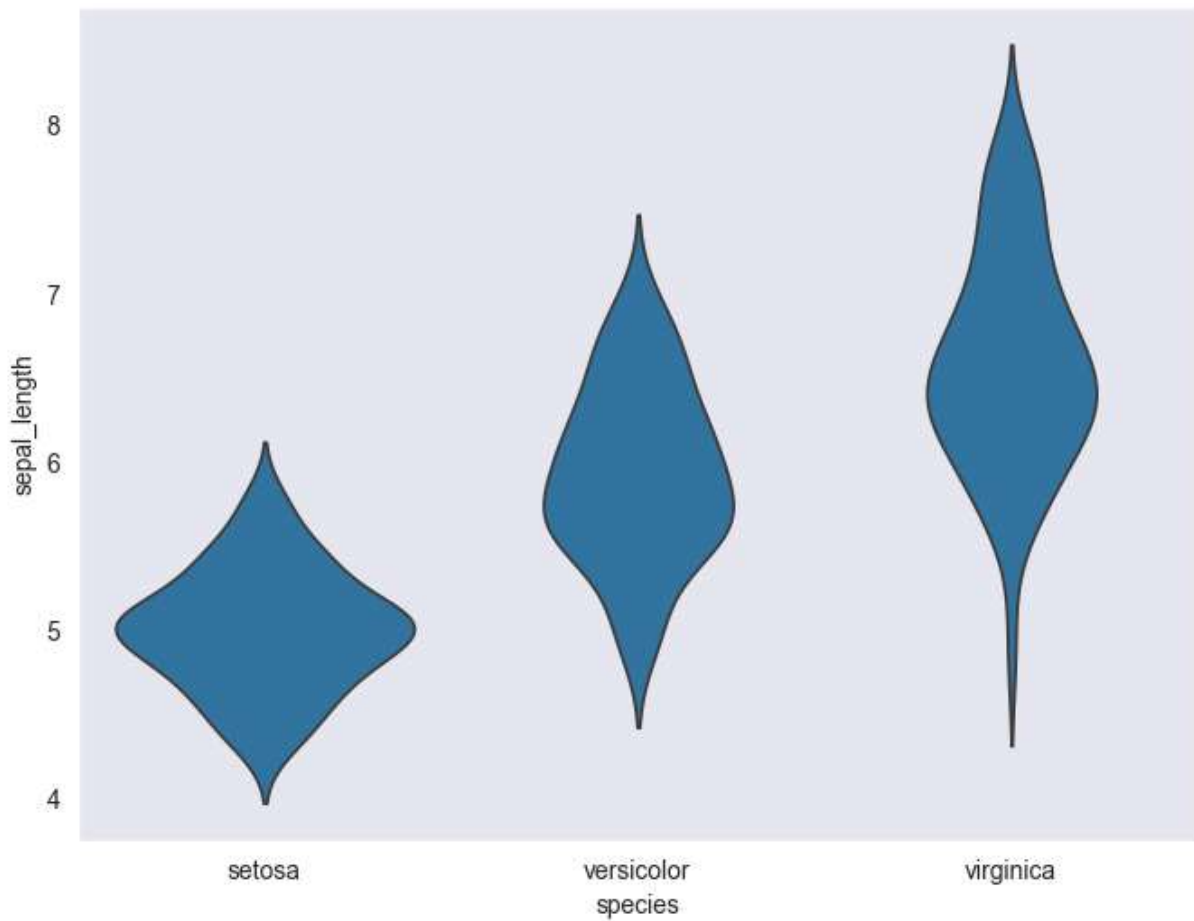
```
In [43]: plt.figure(figsize=(10,8));  
sns.violinplot(x='species',y='sepal_length',data=iris,split=True)
```

```
Out[43]: <Axes: xlabel='species', ylabel='sepal_length'>
```



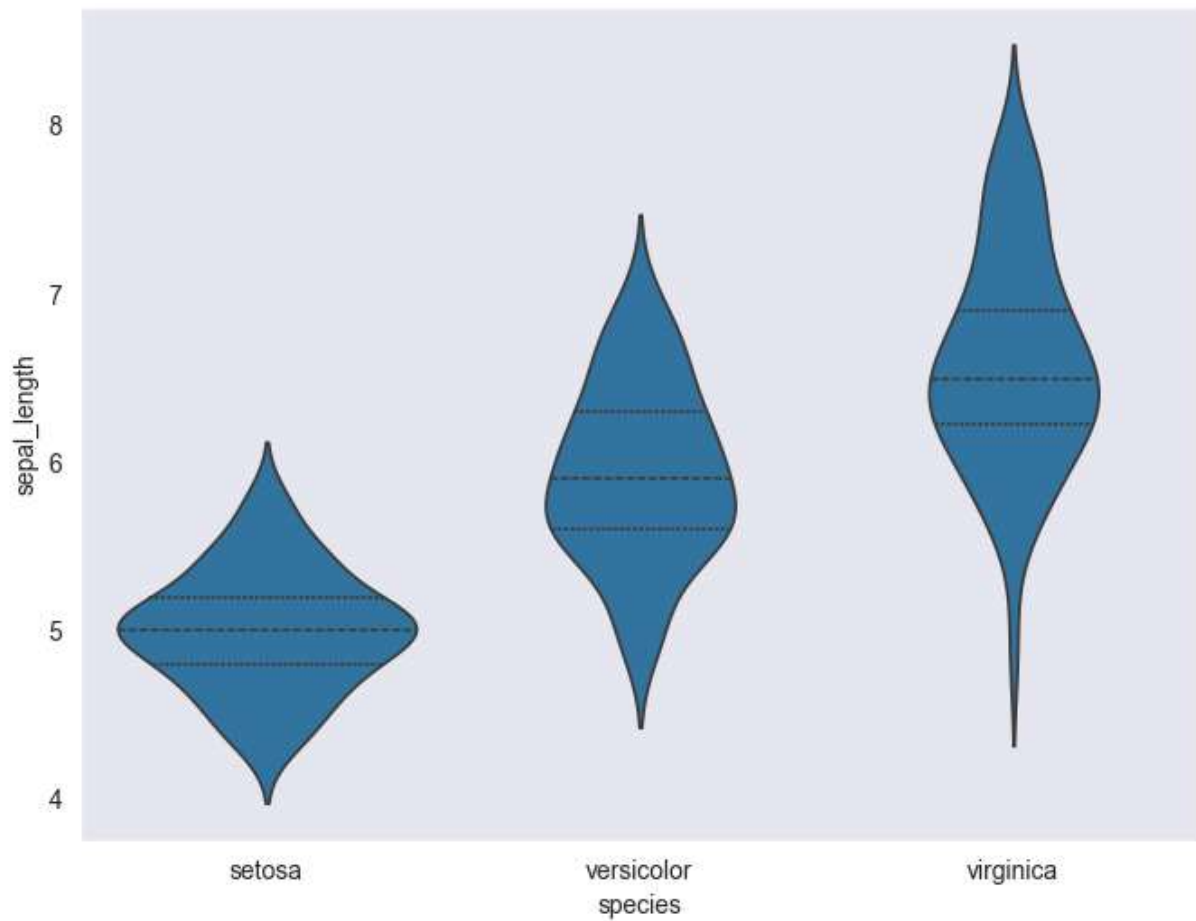
```
In [44]: plt.figure(figsize=(8,6));  
sns.violinplot(x='species',y='sepal_length',data=iris,inner=None)
```

```
Out[44]: <Axes: xlabel='species', ylabel='sepal_length'>
```



```
In [45]: plt.figure(figsize=(8,6));  
sns.violinplot(x='species',y='sepal_length',data=iris,inner='quartile')
```

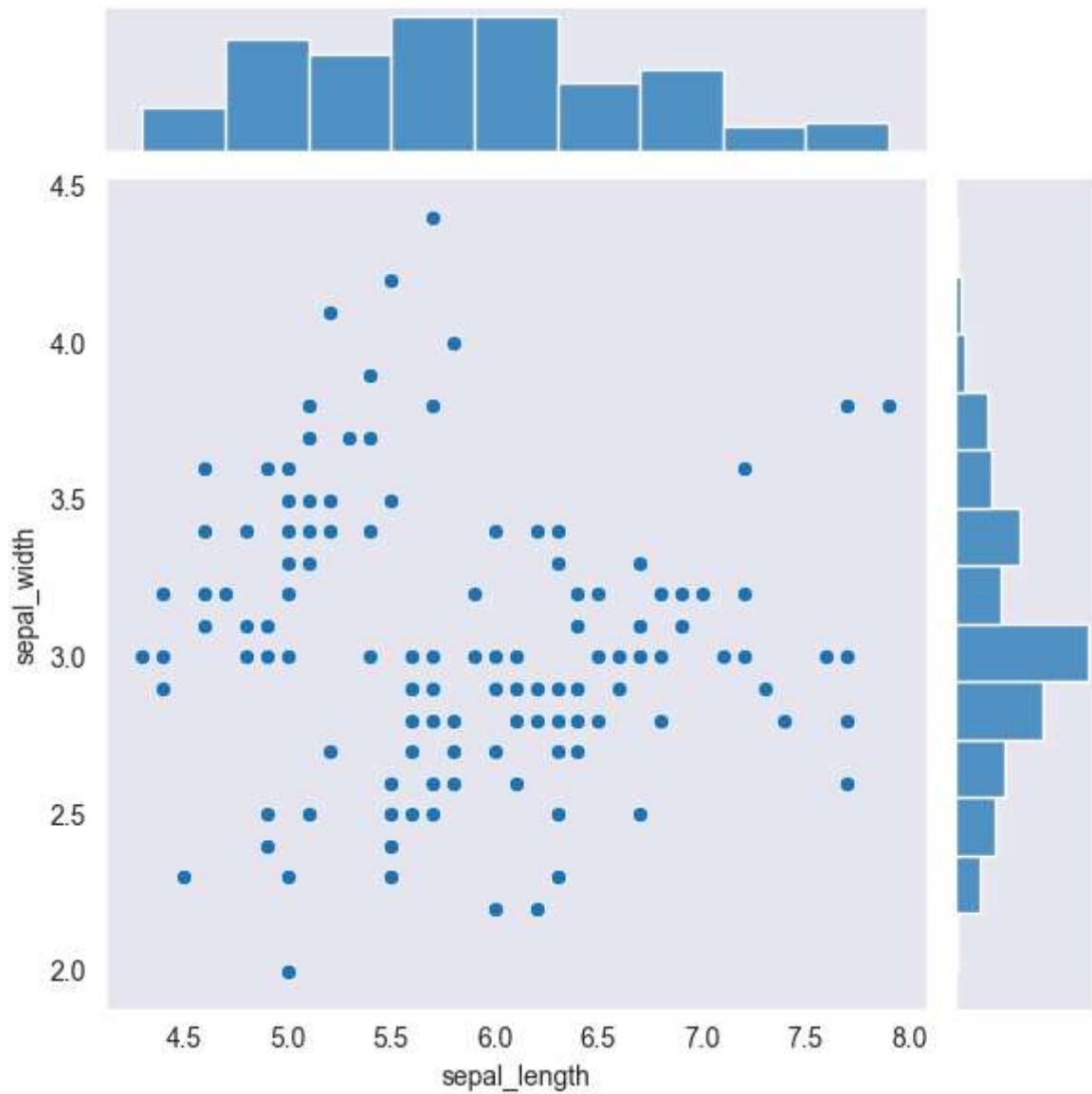
```
Out[45]: <Axes: xlabel='species', ylabel='sepal_length'>
```



comparison plots

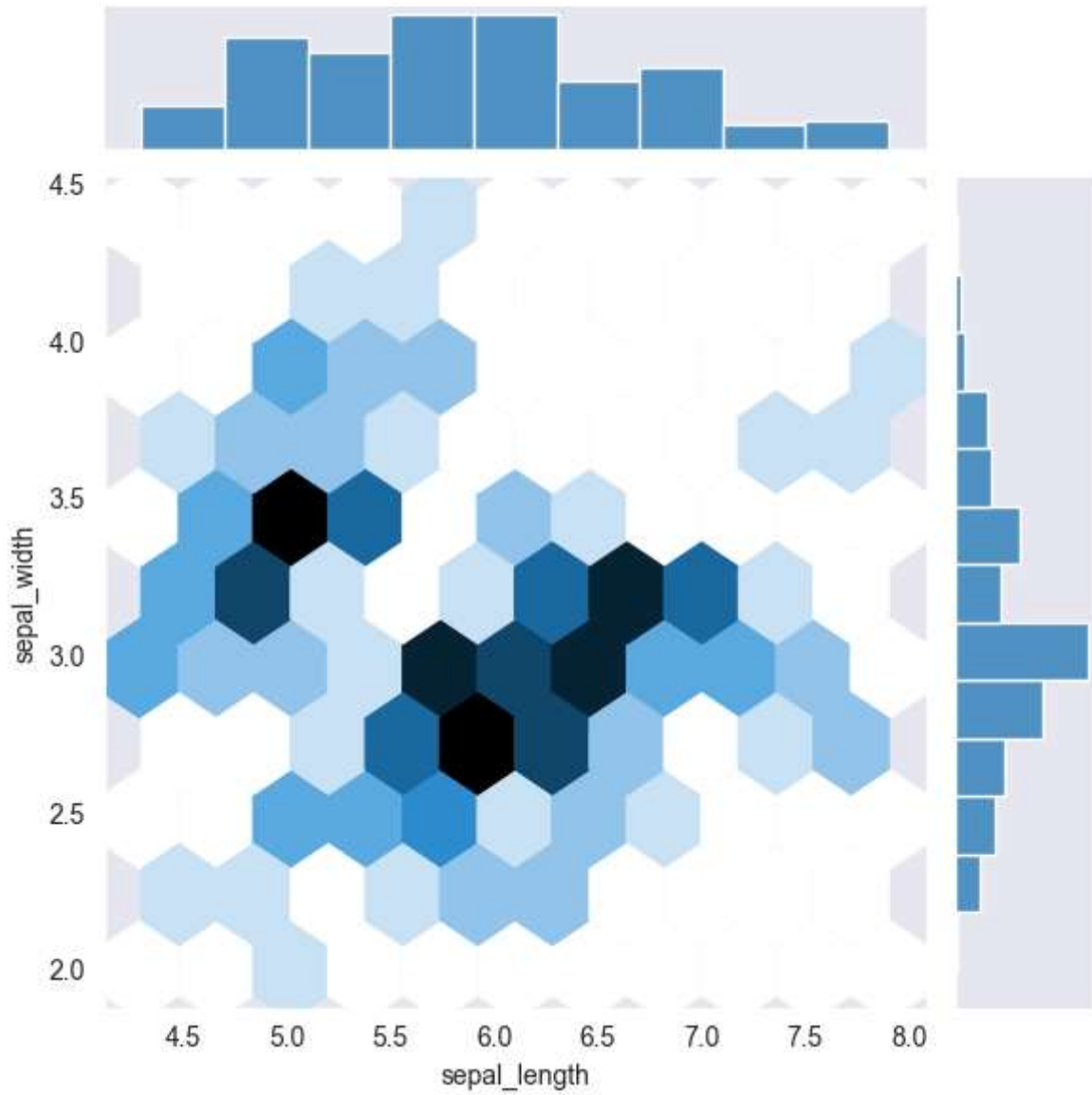
```
In [46]: sns.jointplot(x='sepal_length',y='sepal_width',data=iris)
```

```
Out[46]: <seaborn.axisgrid.JointGrid at 0x1c3abdfea50>
```



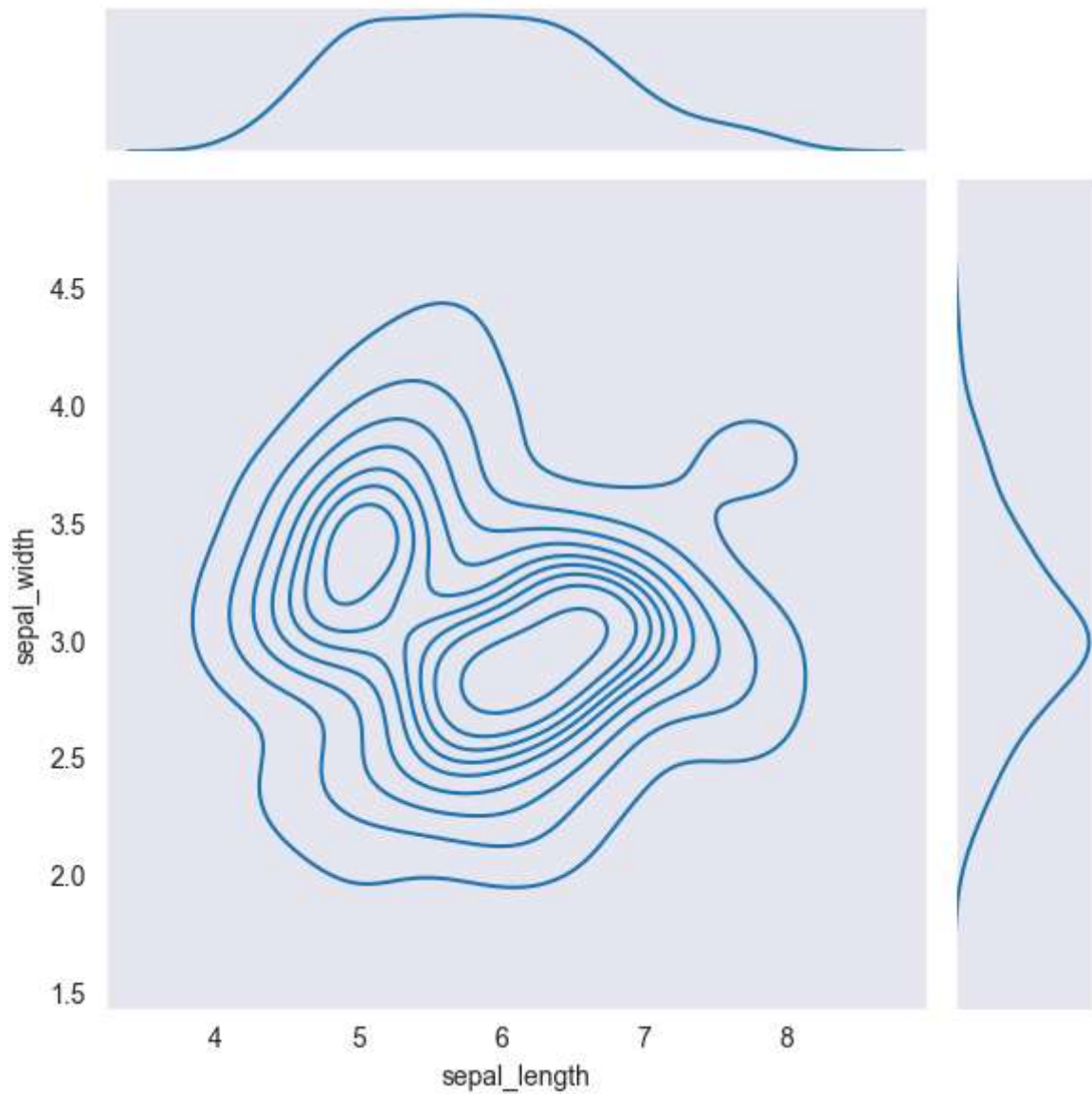
```
In [47]: sns.jointplot(x='sepal_length',y='sepal_width',data=iris,kind="hex")
```

```
Out[47]: <seaborn.axisgrid.JointGrid at 0x1c3b4033750>
```



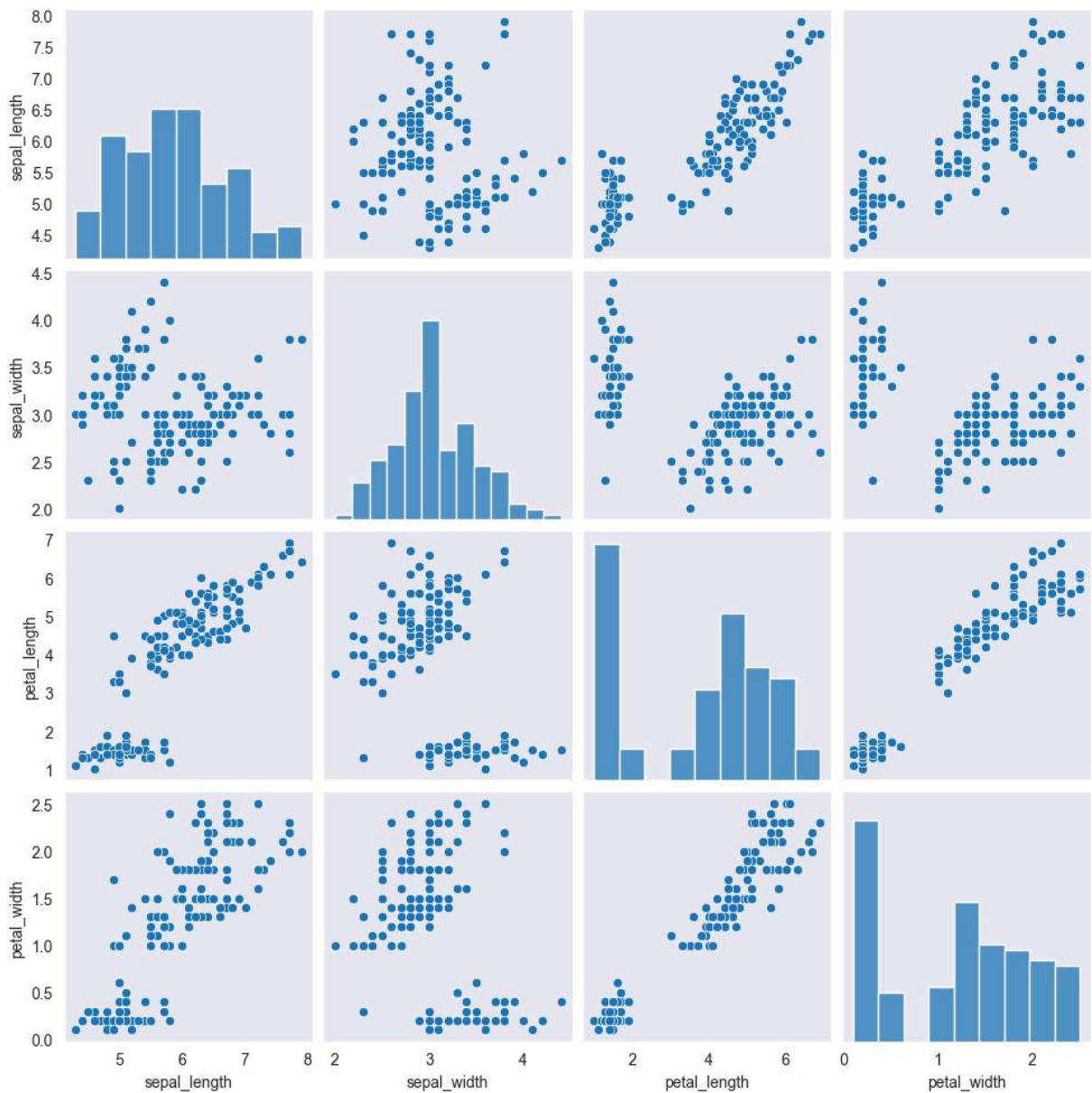
```
In [48]: sns.jointplot(x='sepal_length',y='sepal_width',data=iris,kind="kde")
```

```
Out[48]: <seaborn.axisgrid.JointGrid at 0x1c3b4fb3b10>
```

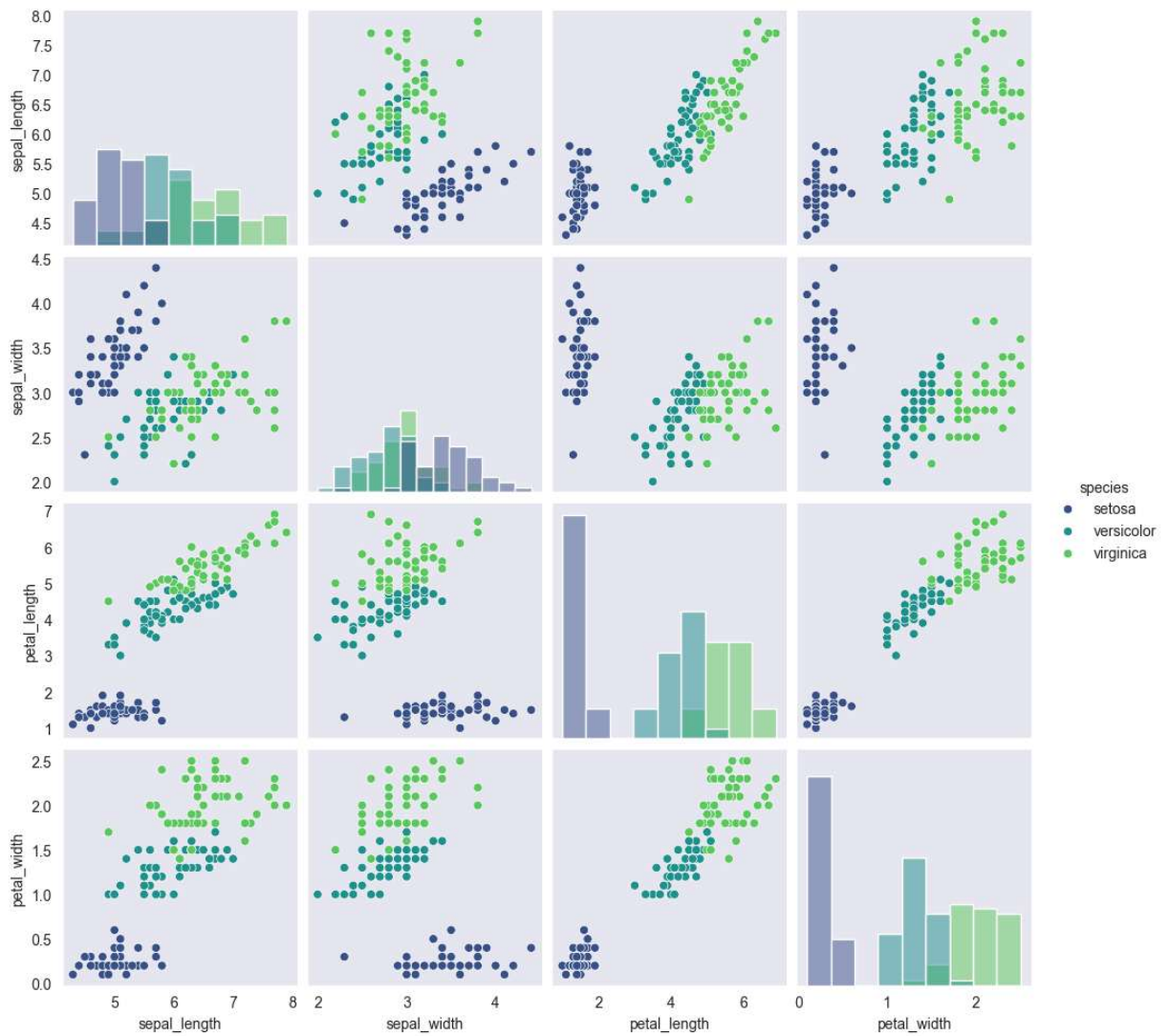
```
In [49]: sns.pairplot(iris)
```

```
Out[49]: <seaborn.axisgrid.PairGrid at 0x1c3b539c2f0>
```



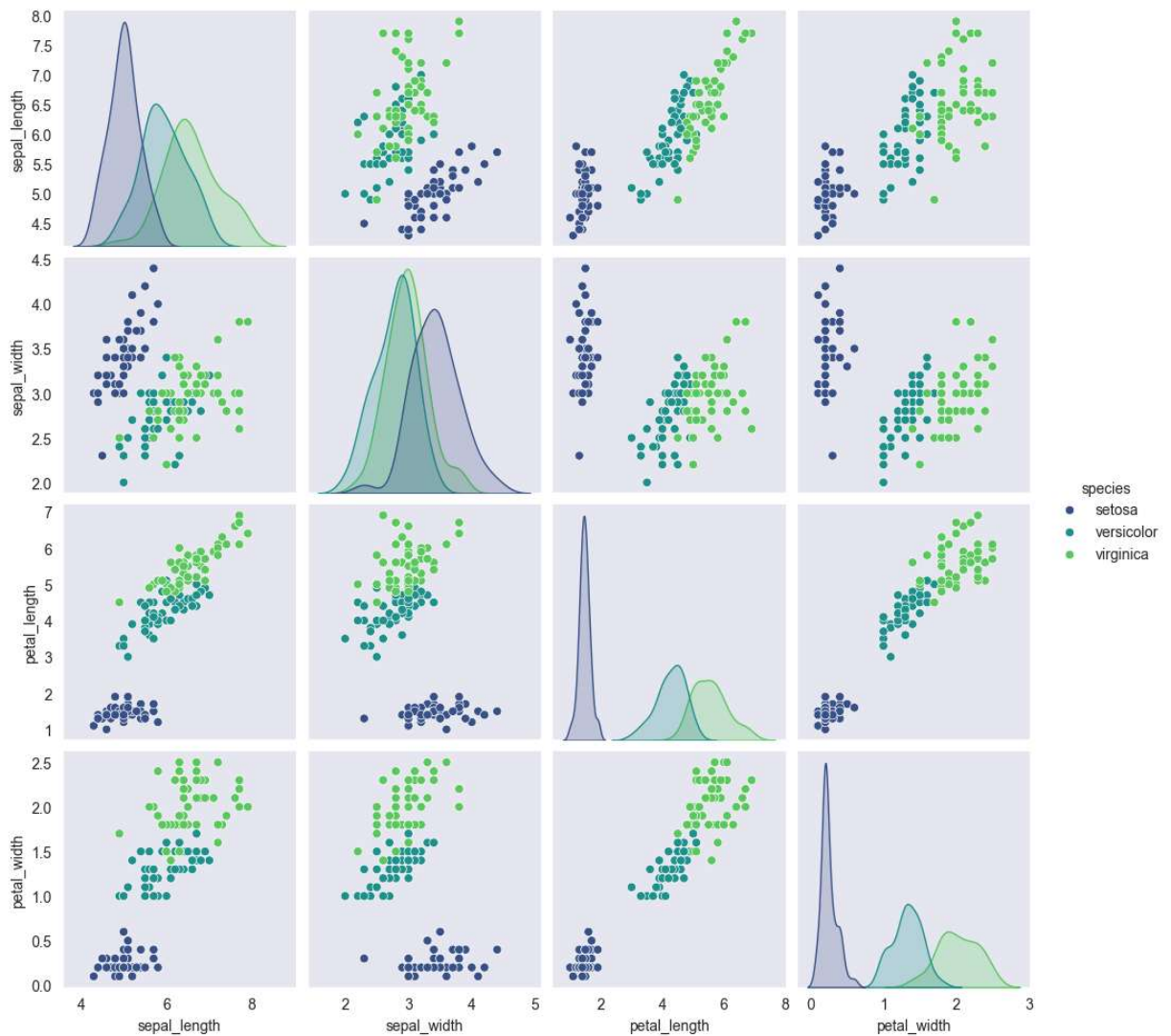
```
In [50]: sns.pairplot(iris,hue='species',palette='viridis',diag_kind='hist')
```

```
Out[50]: <seaborn.axisgrid.PairGrid at 0x1c3b6069a90>
```



```
In [51]: sns.pairplot(iris,hue='species',palette='viridis')
```

```
Out[51]: <seaborn.axisgrid.PairGrid at 0x1c3b8007c50>
```

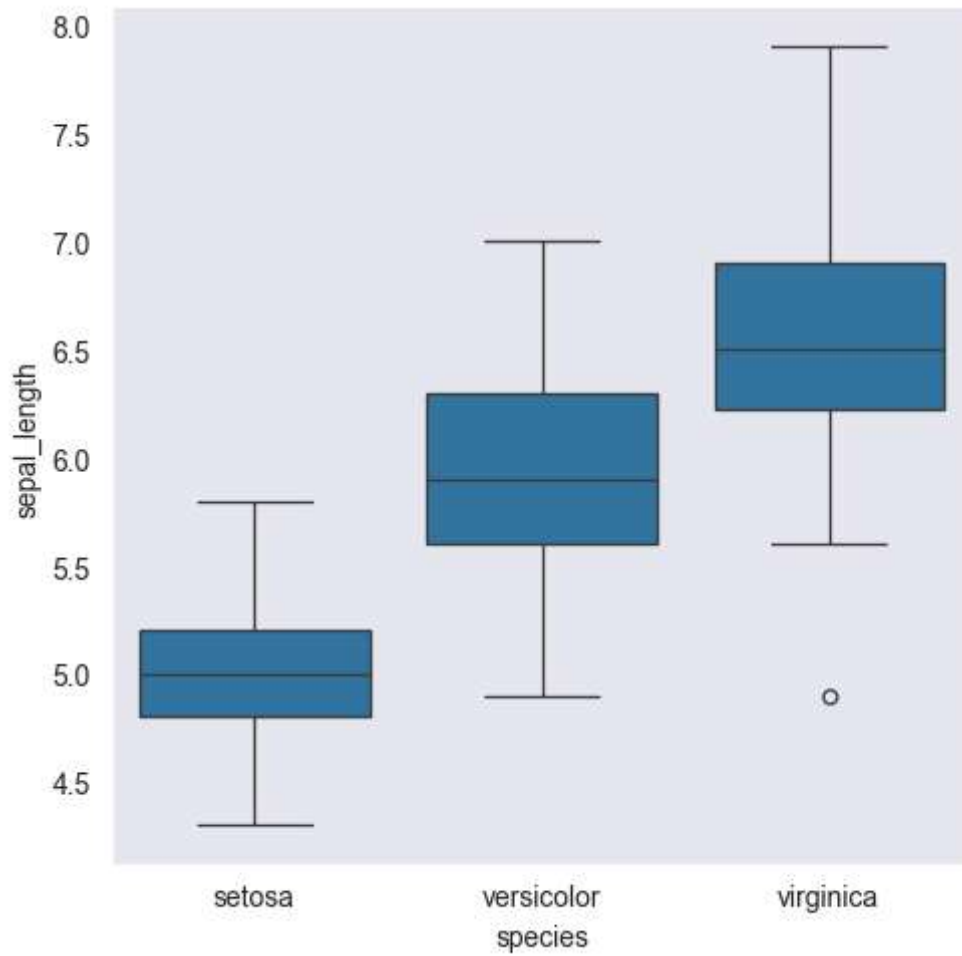


seaborn grids

we use catplot for categorical plots

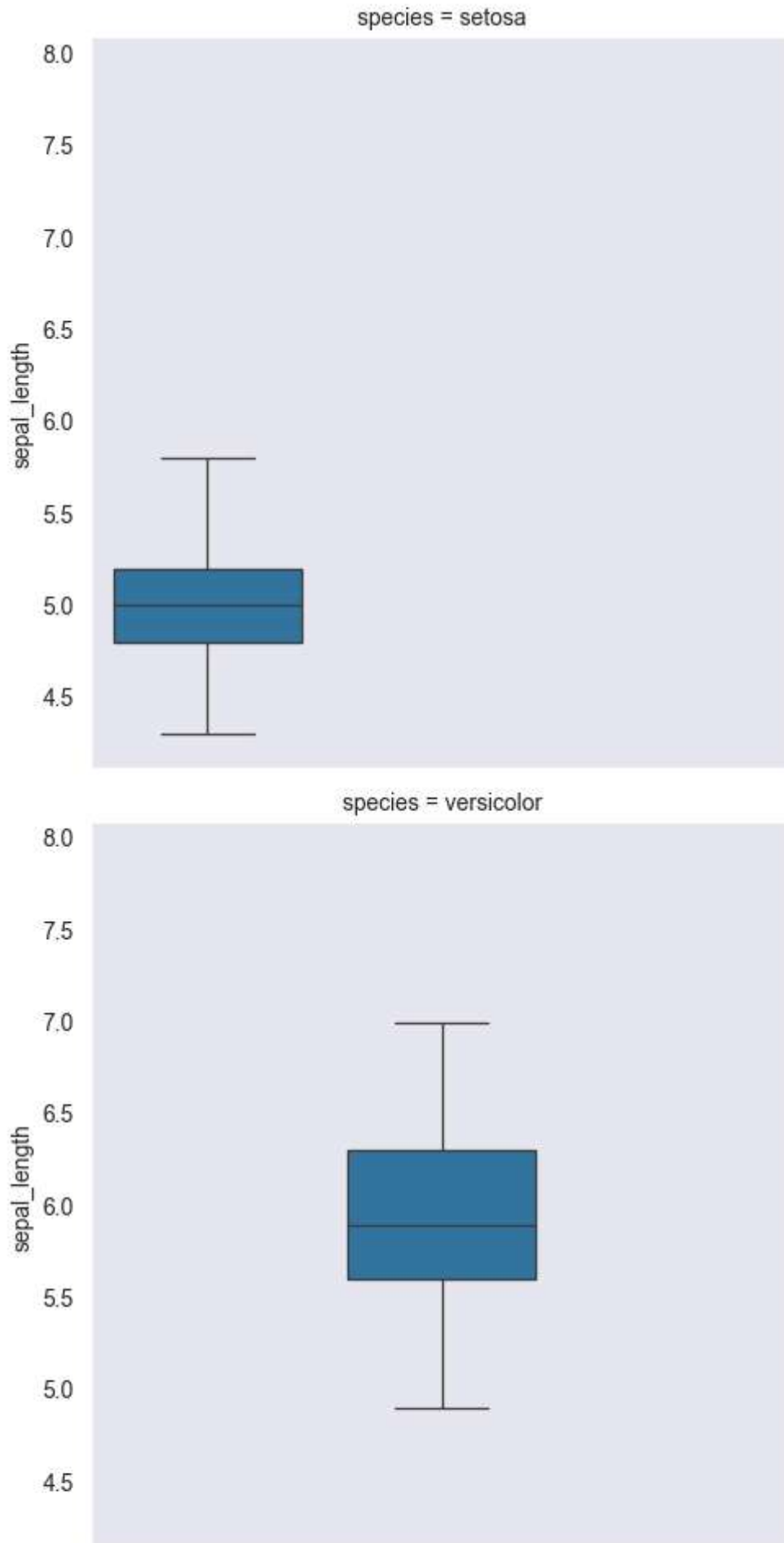
```
In [52]: plt.figure(figsize=(8,6))
sns.catplot(y='sepal_length',x='species',data=iris,kind='box')
plt.show()
```

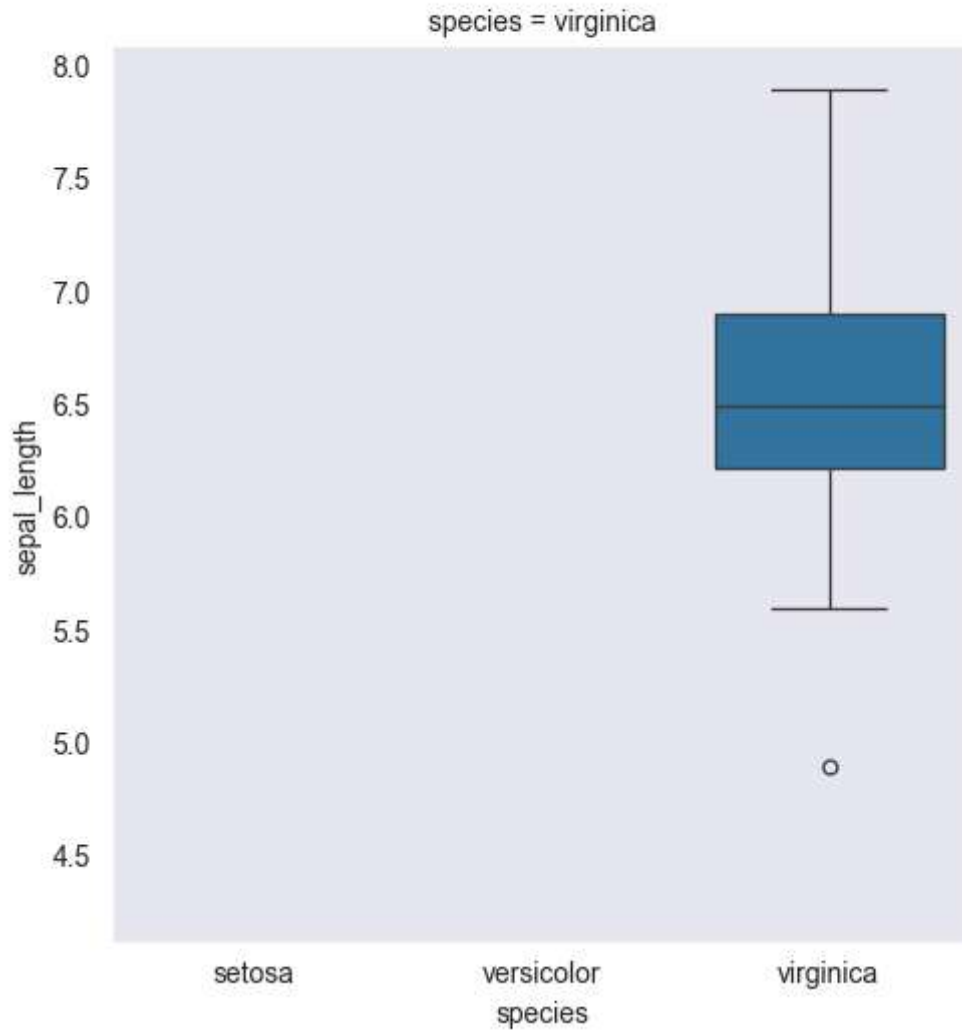
<Figure size 800x600 with 0 Axes>



```
In [53]: plt.figure(figsize=(8,6))
sns.catplot(y='sepal_length',x='species',data=iris,kind='box',row='species')
plt.show()
```

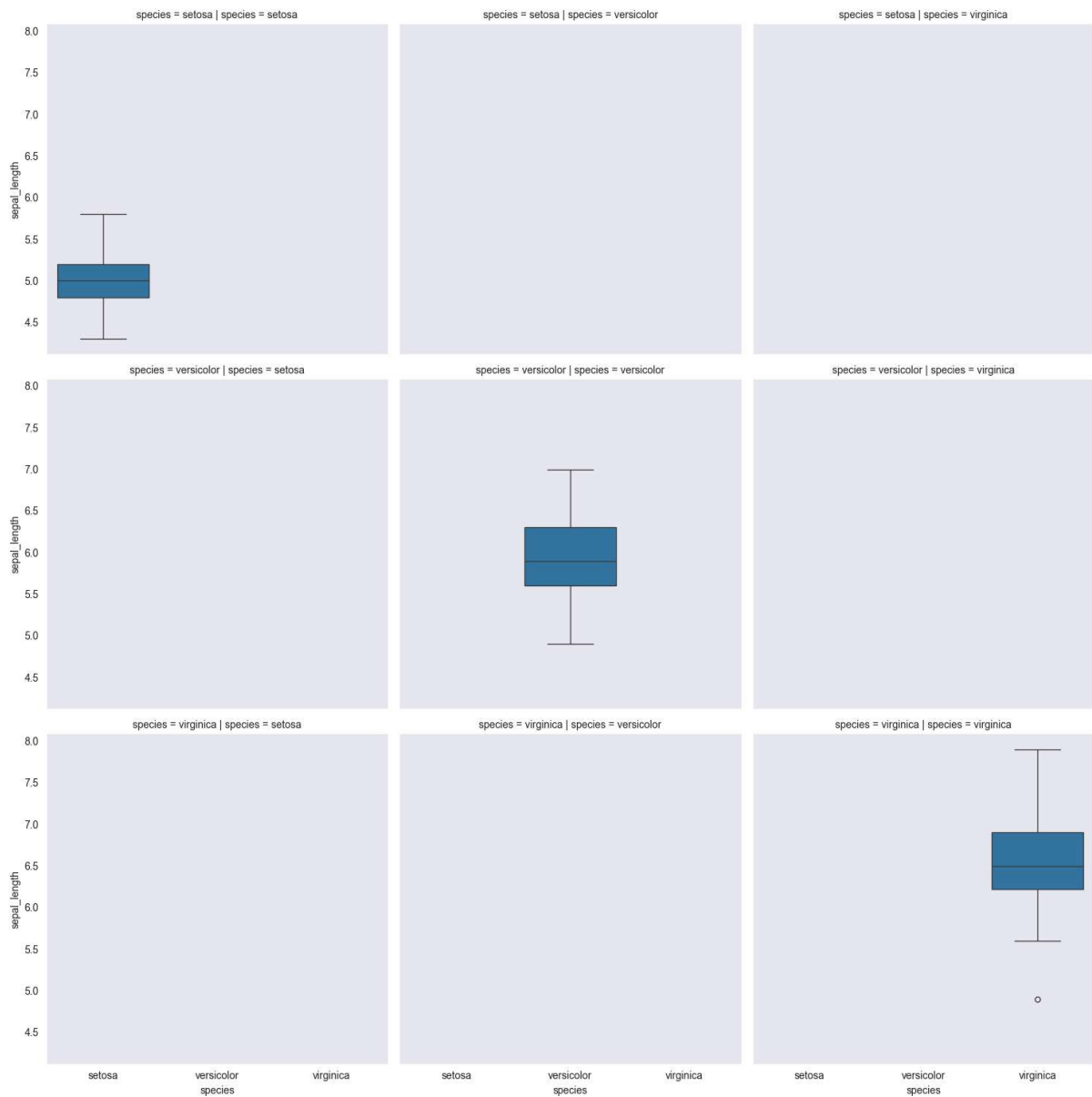
<Figure size 800x600 with 0 Axes>



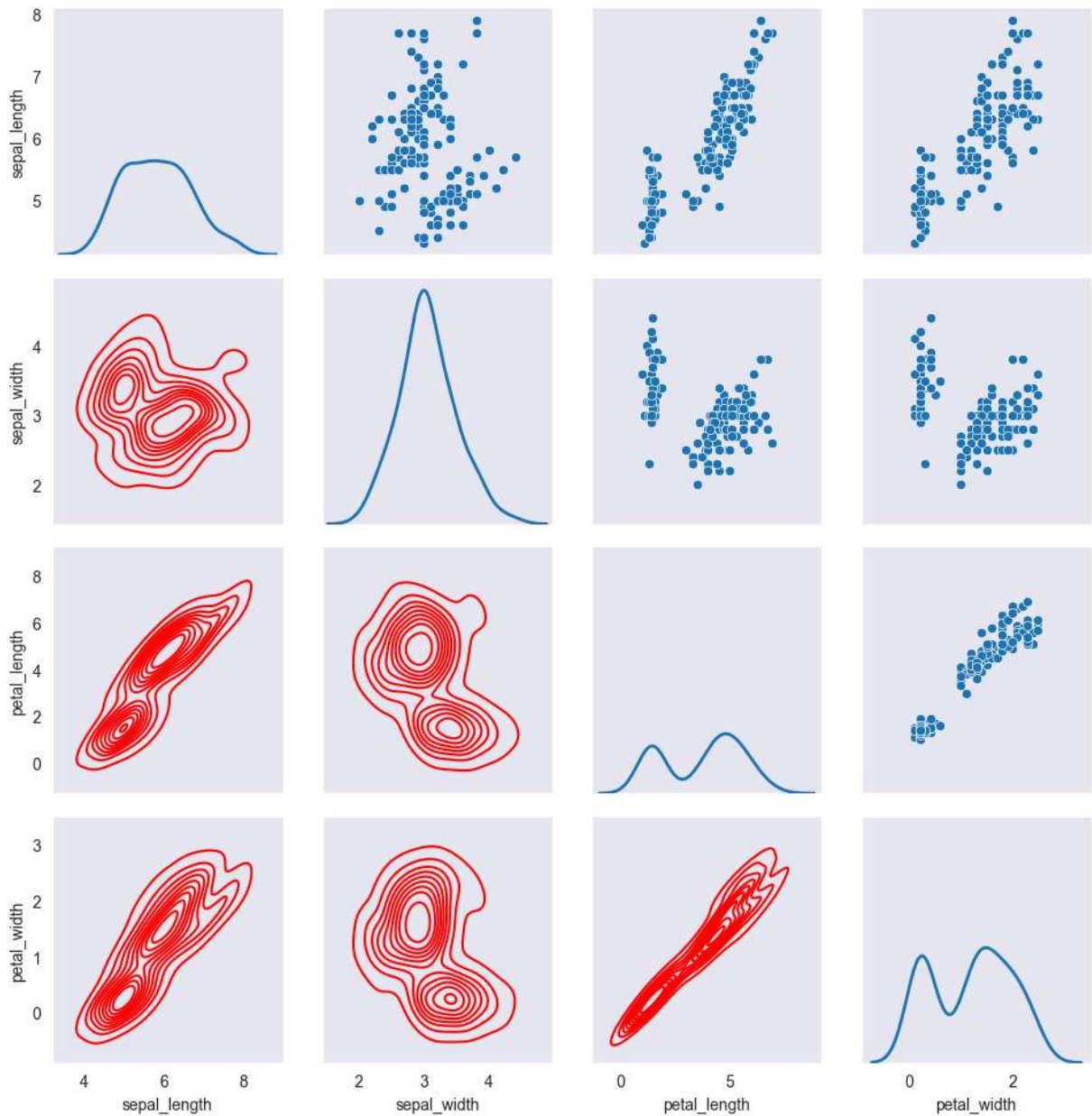


```
In [54]: plt.figure(figsize=(8,6))
sns.catplot(y='sepal_length',x='species',data=iris,kind='box',row='species',col='sp
plt.show()
```

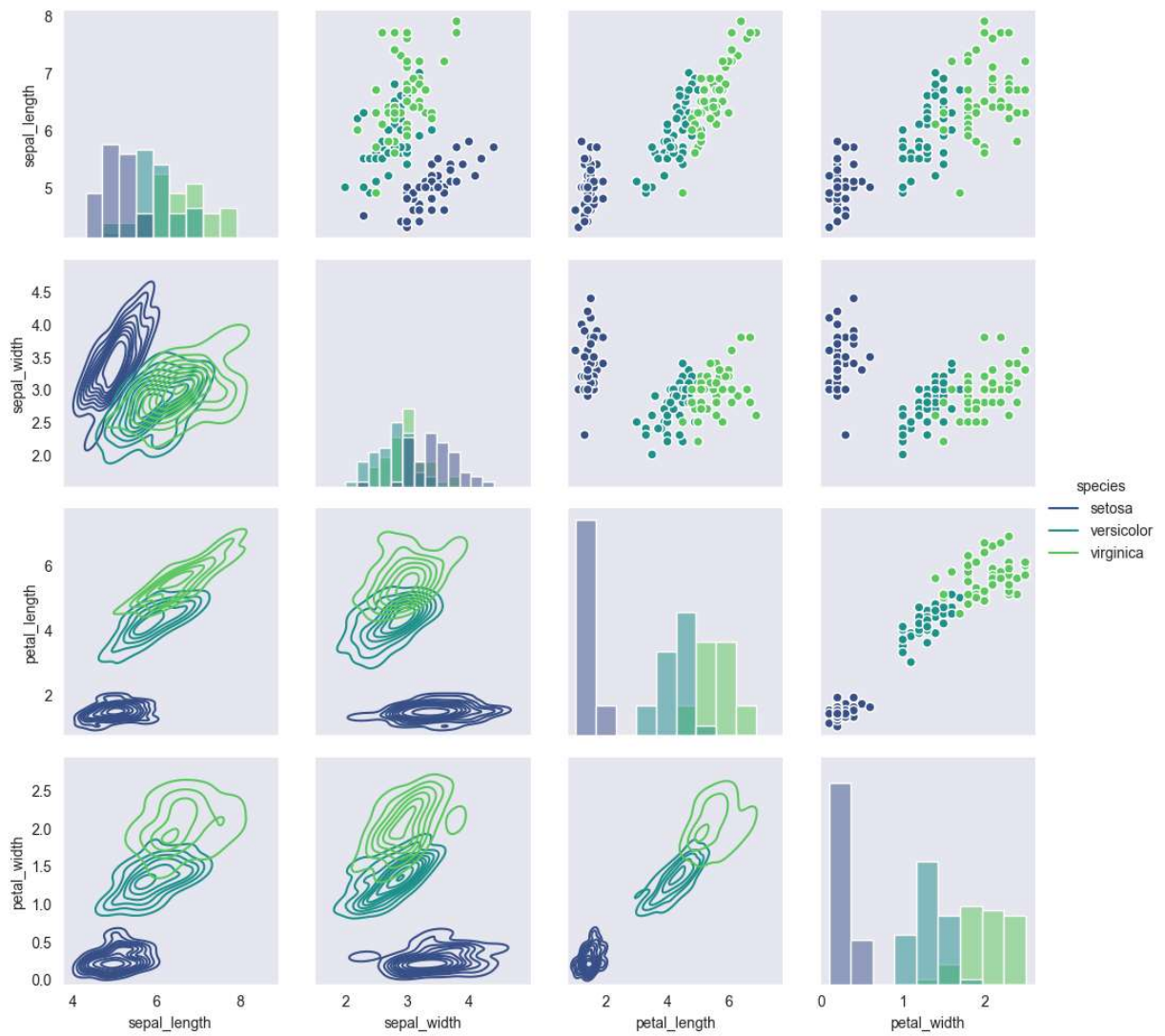
<Figure size 800x600 with 0 Axes>



```
In [55]: g=sns.PairGrid(iris)
g=g.map_upper(sns.scatterplot)
g=g.map_diag(sns.kdeplot,lw=2)
g=g.map_lower(sns.kdeplot,colors='red')
```

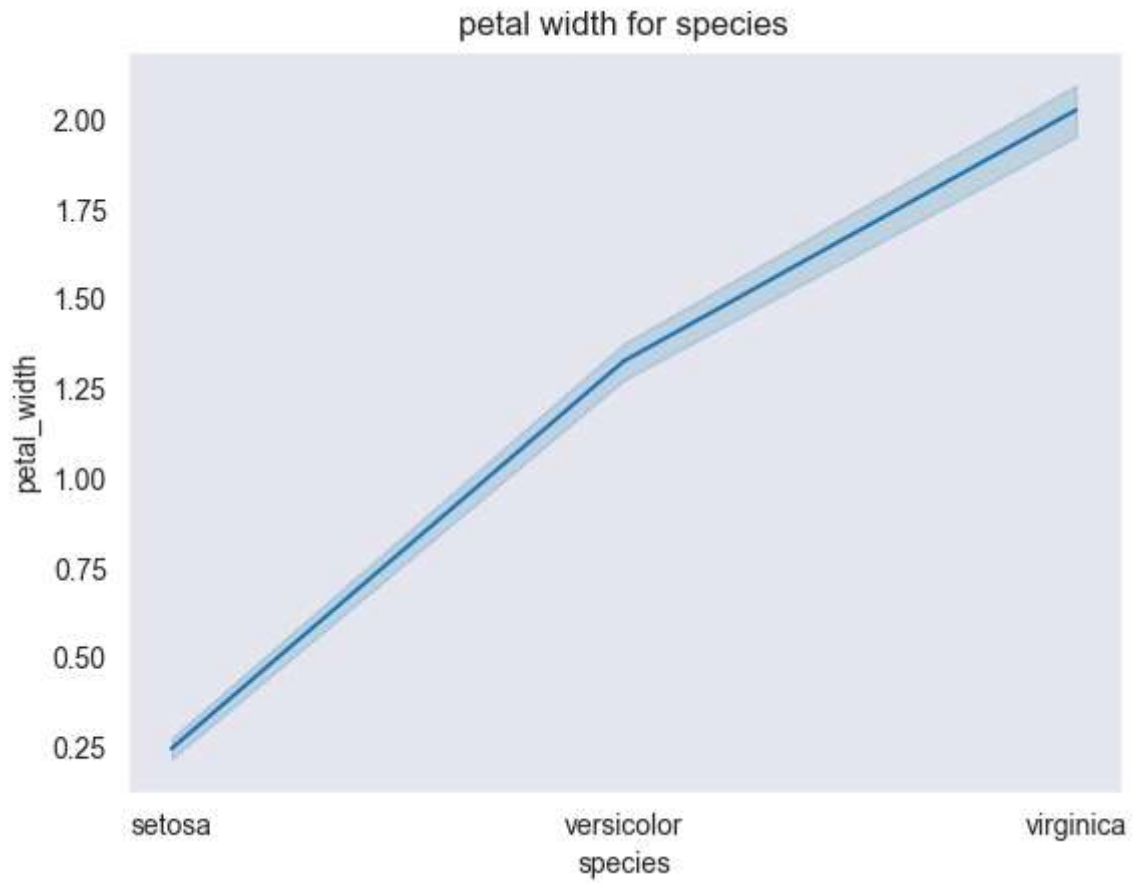



```
In [56]: g=sns.PairGrid(iris,hue='species',palette='viridis')
g=g.map_upper(sns.scatterplot,linewidths=1,edgecolor='w',s=40)
g=g.map_diag(sns.histplot)
g=g.map_lower(sns.kdeplot)
g=g.add_legend();
```

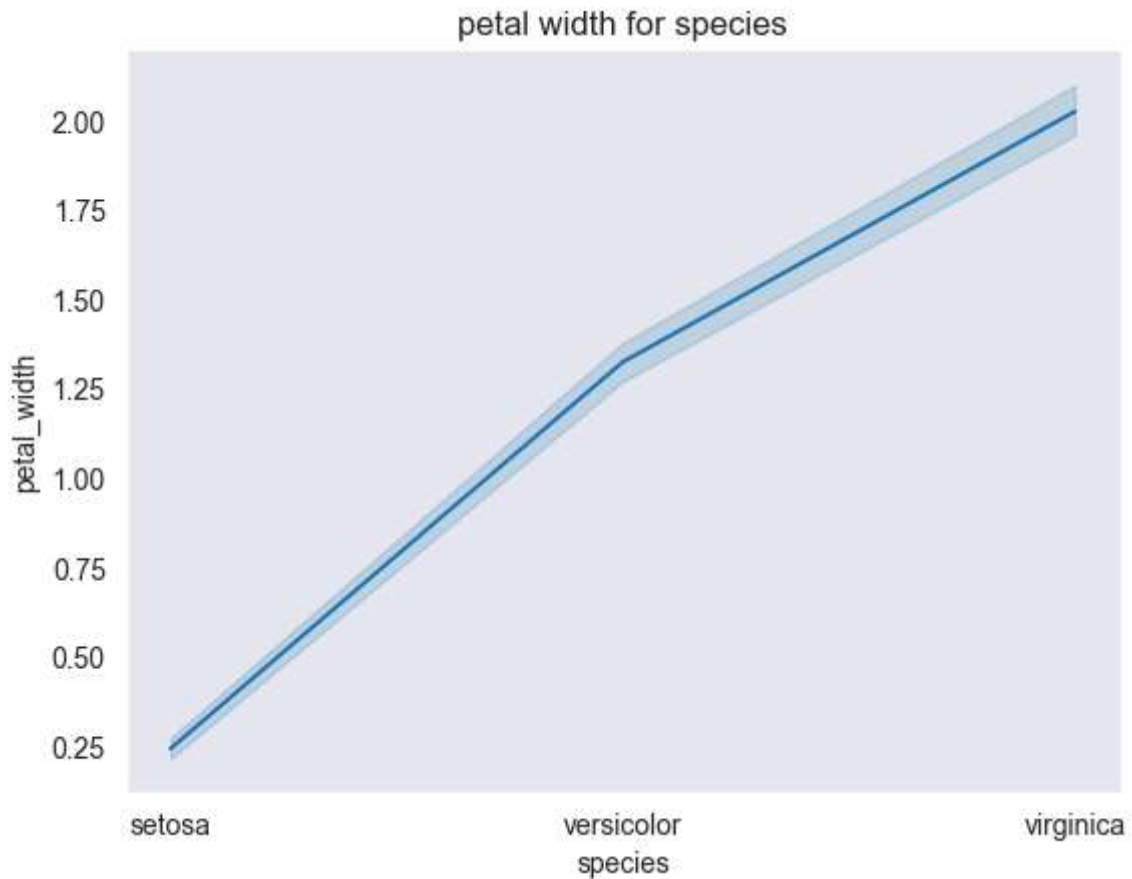


using seaborn with matplotlib

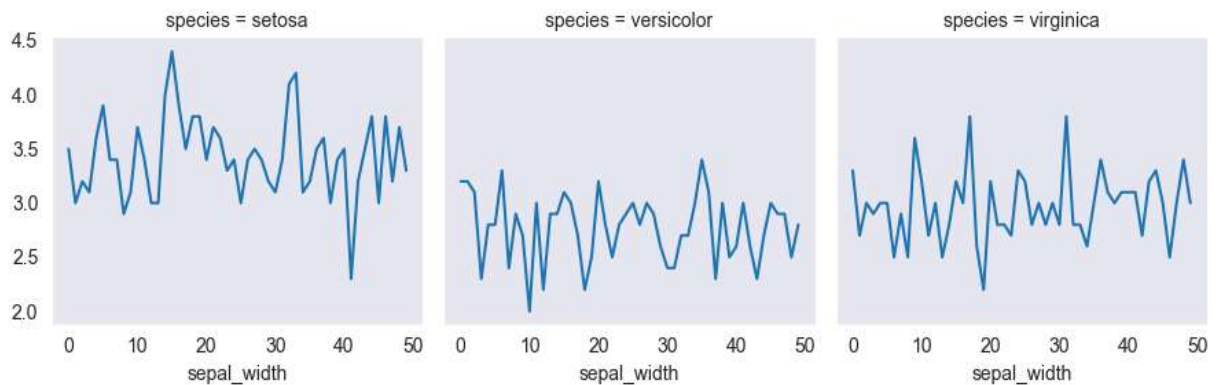
```
In [57]: sns.lineplot(x="species",y="petal_width",data=iris)
plt.title("petal width for species")
plt.show()
```



```
In [59]: sns.lineplot(x="species",y="petal_width",data=iris)
plt.title("petal width for species")
sns.despine()
plt.show()
```



```
In [61]: plot=sns.FacetGrid(iris,col="species")
plot.map(plt.plot,"sepal_width")
plt.show()
```



```
In [64]: tc = iris.select_dtypes(include='number').corr() # keep only numeric columns
sns.heatmap(tc, annot=True, cmap="coolwarm")
plt.show()
```

